

Pemrograman Android dengan Kotlin



 github.com/ocikyamin



Prepared By :
Abdul Yamin, S.Pd., M.Kom

Pengantar

Modul ini dirancang untuk membimbing Anda dalam perjalanan menjadi pengembang aplikasi Android yang kompeten menggunakan bahasa pemrograman Kotlin. Selama satu semester, Anda akan mempelajari konsep-konsep fundamental hingga praktik terbaik dalam membangun aplikasi Android yang responsif, fungsional, dan siap distribusi. Modul ini akan mencakup teori mendalam, contoh kode praktis, dan langkah-langkah praktikum yang akan membantu Anda menguasai setiap keterampilan yang ditargetkan.

Target Capaian Keterampilan Khusus

Setelah menyelesaikan modul ini, Anda diharapkan mampu:

1. **Membuat antarmuka pengguna (UI) yang responsif** menggunakan berbagai komponen layout dan widget di Android Studio dengan XML.
2. **Mengimplementasikan logika bisnis aplikasi** menggunakan dasar-dasar pemrograman Kotlin seperti variabel, percabangan, dan penanganan event.
3. **Mengelola navigasi aplikasi dan komunikasi antar layar (Activity)** menggunakan Intent dan AndroidManifest.
4. **Mengimplementasikan penyimpanan data lokal** dengan Room Database dan pertukaran data dengan server melalui RESTful API.
5. **Berkolaborasi dalam pengembangan aplikasi lengkap** hingga menghasilkan file APK yang siap distribusi.



Struktur Modul

Modul ini akan dibagi menjadi beberapa bagian utama, masing-masing berfokus pada target capaian keterampilan tertentu. Setiap bagian akan mencakup teori, contoh kode, dan praktikum.

Bagian 1: Fondasi Android dan Kotlin

- **Minggu 1-2:** Persiapan Lingkungan Pengembangan dan Dasar-dasar Kotlin
 - Instalasi Android Studio dan JDK.
 - Pengenalan Kotlin: Variabel, Tipe Data, Operator, Struktur Kontrol, Fungsi, Null Safety.
 - Membuat proyek Android pertama.

Bagian 2: Membangun Antarmuka Pengguna (UI) Responsif

- **Minggu 3-4:** Komponen UI Dasar dan Layouts
 - `TextView`, `Button`, `EditText`, `ImageView`.
 - `LinearLayout`, `RelativeLayout`, `ConstraintLayout`, `FrameLayout`, `GridLayout`.
 - Properti UI umum: `padding`, `margin`, `gravity`, `visibility`.
 - Membangun UI yang adaptif untuk berbagai ukuran layar.

Bagian 3: Logika Bisnis dan Interaksi Pengguna

- **Minggu 5-6:** Kotlin Lanjutan dan Penanganan Event
 - Pemrograman Berorientasi Objek (OOP) di Kotlin: Class, Object, Inheritance, Interface, Data Class.
 - Collections: List, Set, Map.
 - Lambda Expressions dan Higher-Order Functions.
 - Penanganan event UI yang kompleks: `OnClickListener`, `OnLongClickListener`, `TextWatcher`.
 - Pengenalan `ViewModel` dan `LiveData` untuk manajemen data UI.

Bagian 4: Navigasi dan Struktur Aplikasi

- **Minggu 7-8:** Activity, Intent, dan Fragment
 - Activity Lifecycle secara mendalam.
 - `Explicit Intent` dan `Implicit Intent` untuk navigasi dan komunikasi antar aplikasi.
 - Mengirim dan menerima data antar Activity.
 - Pengenalan `Fragment` dan `FragmentManager` untuk UI yang modular.
 - Navigasi menggunakan Navigation Component.

Bagian 5: Penyimpanan Data Lokal

- **Minggu 9-10:** Room Database
 - Pengenalan konsep database lokal di Android.
 - Arsitektur Room: `Entity`, `DAO` (Data Access Object), `Database`.

- Operasi CRUD (Create, Read, Update, Delete) dengan Room.
- Integrasi Room dengan `LiveData` atau `Flow` untuk data reaktif.

Bagian 6: Konektivitas Jaringan dan RESTful API

- **Minggu 11-12:** Mengonsumsi RESTful API
 - Pengenalan konsep RESTful API dan format data JSON.
 - Penggunaan library Retrofit untuk melakukan HTTP requests.
 - Parsing JSON responses menggunakan GSON atau `kotlinx.serialization`.
 - Menampilkan data dari API ke UI (misalnya, menggunakan `RecyclerView`).
 - Penanganan error jaringan.

Bagian 7: Kolaborasi dan Distribusi

- **Minggu 13-14:** Version Control dan Build Aplikasi
 - Pengenalan Git dan GitHub untuk kolaborasi tim.
 - Branching, merging, pull requests.
 - Proses build aplikasi Android: Debug vs Release APK.
 - Signing aplikasi untuk distribusi di Google Play Store.
 - Persiapan untuk rilis aplikasi.

Bagian 1: Fondasi Android dan Kotlin

Tujuan Pembelajaran

Setelah menyelesaikan bagian ini, Anda diharapkan mampu:

- Menginstal dan mengkonfigurasi Android Studio dan JDK.
- Memahami dasar-dasar bahasa pemrograman Kotlin, termasuk variabel, tipe data, operator, struktur kontrol, fungsi, dan null safety.
- Membuat proyek Android pertama dan memahami strukturnya.
- Menjalankan aplikasi Android di emulator atau perangkat fisik.

1.1 Persiapan Lingkungan Pengembangan

Sebelum memulai pengembangan aplikasi Android, Anda perlu menyiapkan lingkungan pengembangan yang sesuai. Alat utama yang akan kita gunakan adalah Android Studio, Integrated Development Environment (IDE) resmi untuk pengembangan Android.

1.1.1 Instalasi Java Development Kit (JDK)

Android Studio membutuhkan Java Development Kit (JDK) untuk berfungsi dengan baik. Pastikan Anda memiliki JDK versi 11 atau yang lebih baru terinstal di sistem Anda. Anda dapat mengunduh JDK dari situs web Oracle atau menggunakan OpenJDK.

Langkah-langkah Instalasi (Contoh untuk OpenJDK):

1. **Unduh OpenJDK:** Kunjungi situs web OpenJDK (misalnya, Adoptium atau Oracle OpenJDK) dan unduh versi yang sesuai untuk sistem operasi Anda.
2. **Instalasi:** Ikuti instruksi instalasi yang diberikan untuk sistem operasi Anda (Windows, macOS, Linux).
3. **Verifikasi Instalasi:** Buka terminal atau Command Prompt dan ketik:

```
java -version  
javac -version
```

Anda akan melihat versi Java yang terinstal jika instalasi berhasil.

1.1.2 Instalasi Android Studio

Android Studio adalah IDE yang kuat yang menyediakan semua alat yang Anda butuhkan untuk mengembangkan aplikasi Android, termasuk editor kode, debugger, emulator, dan alat build.

Langkah-langkah Instalasi:

1. **Unduh Android Studio:** Kunjungi situs web resmi Android Studio:
<https://developer.android.com/studio>
2. **Instalasi:** Ikuti instruksi instalasi untuk sistem operasi Anda. Proses instalasi biasanya melibatkan:
 - Menjalankan installer.
 - Menerima perjanjian lisensi.
 - Memilih komponen yang akan diinstal (pastikan **Android SDK**, **Android SDK Platform-Tools**, dan **Android Virtual Device** terpilih).
 - Menentukan lokasi instalasi.
 - Menyelesaikan proses instalasi.
3. **Konfigurasi Awal:** Saat pertama kali membuka Android Studio, Anda mungkin akan diminta untuk mengunduh komponen SDK tambahan. Pastikan koneksi internet Anda stabil.

Verifikasi Instalasi Android Studio:

Setelah instalasi selesai, Anda akan melihat layar selamat datang Android Studio. Ini menandakan bahwa lingkungan pengembangan Anda siap.

1.2 Dasar-dasar Kotlin

Kotlin adalah bahasa pemrograman modern, statis, dan pragmatis yang dikembangkan oleh JetBrains. Ini adalah bahasa pilihan untuk pengembangan Android, menawarkan fitur-fitur yang meningkatkan produktivitas dan keamanan kode dibandingkan Java.

1.2.1 Variabel dan Tipe Data

Kotlin memiliki dua jenis variabel utama:

- **val** (immutable/read-only): Nilainya tidak dapat diubah setelah diinisialisasi.
- **var** (mutable/read-write): Nilainya dapat diubah setelah diinisialisasi.

Kotlin mendukung inferensi tipe, yang berarti Anda tidak selalu perlu secara eksplisit mendeklarasikan tipe data. Namun, Anda bisa melakukannya jika diperlukan.

Tipe Data Dasar:

- **Angka:** **Byte**, **Short**, **Int**, **Long**, **Float**, **Double**
- **Boolean:** **Boolean** (**true** atau **false**)
- **Karakter:** **Char**
- **String:** **String**

Contoh:

```
fun main() {  
    // Variabel immutable (val)  
    val nama: String = "Alice"  
    val umur = 30 // Inferensi tipe: Int  
  
    // Variabel mutable (var)  
    var saldo = 1000.0 // Inferensi tipe: Double  
    saldo = 1500.0  
  
    println("Nama: $nama, Umur: $umur")  
    println("Saldo: $saldo")  
  
    // Tipe data eksplisit  
    val isStudent: Boolean = true  
    val initial: Char = 'A'  
}
```

1.2.2 Operator

Kotlin mendukung operator aritmatika, perbandingan, logika, dan penugasan yang umum.

Contoh:

```
fun main() {  
    val a = 10  
    val b = 5  
  
    // Aritmatika  
    println("Penjumlahan: ${a + b}")  
    println("Pengurangan: ${a - b}")  
    println("Perkalian: ${a * b}")  
    println("Pembagian: ${a / b}")  
    println("Modulus: ${a % b}")  
  
    // Perbandingan  
    println("a > b: ${a > b}")  
    println("a == b: ${a == b}")  
  
    // Logika
```

```
    val isTrue = true
    val isFalse = false
    println("isTrue && isFalse: ${isTrue && isFalse}")
    println("isTrue || isFalse: ${isTrue || isFalse}")
    println("!isTrue: ${!isTrue}")
}
```

1.2.3 Struktur Kontrol

Kotlin menyediakan struktur kontrol untuk mengontrol alur eksekusi program.

If-Else Expression:

```
fun main() {
    val nilai = 75
    val hasil = if (nilai >= 70) {
        "Lulus"
    } else {
        "Tidak Lulus"
    }
    println("Hasil: $hasil")
}
```

When Expression (Mirip Switch-Case):

```
fun main() {
    val hari = 3
    val namaHari = when (hari) {
        1 -> "Minggu"
        2 -> "Senin"
        3 -> "Selasa"
        4 -> "Rabu"
        5 -> "Kamis"
        6 -> "Jumat"
        7 -> "Sabtu"
        else -> "Hari tidak valid"
    }
}
```

```
println("Hari ini adalah: $namaHari")
}
```

For Loop:

```
fun main() {
    val angka = listOf(1, 2, 3, 4, 5)
    for (i in angka) {
        println("Angka: $i")
    }

    for (i in 1..5) { // Range
        println("Iterasi: $i")
    }
}
```

While Loop:

```
fun main() {
    var hitung = 0
    while (hitung < 5) {
        println("Hitung: $hitung")
        hitung++
    }
}
```

1.2.4 Fungsi

Fungsi adalah blok kode yang melakukan tugas tertentu. Kotlin mendukung fungsi dengan parameter, nilai kembalian, dan fungsi satu baris.

Contoh:

```
fun sapa(nama: String) { // Fungsi tanpa nilai kembalian (Unit)
    println("Halo, $nama!")
}
```

```

}

fun tambah(a: Int, b: Int): Int { // Fungsi dengan nilai kembalian Int
    return a + b
}

fun kali(a: Int, b: Int) = a * b // Fungsi satu baris

fun main() {
    sapa("Budi")
    val hasilTambah = tambah(5, 3)
    println("Hasil tambah: $hasilTambah")
    val hasilKali = kali(4, 2)
    println("Hasil kali: $hasilKali")
}

```

1.2.5 Null Safety

Salah satu fitur paling menonjol di Kotlin adalah null safety, yang membantu menghilangkan `NullPointerException` yang sering terjadi di Java. Secara default, variabel di Kotlin tidak boleh null.

- **Non-nullable types:** Variabel tidak dapat menyimpan nilai null.
- **Nullable types:** Variabel dapat menyimpan nilai null, ditandai dengan `?` setelah tipe data.

Operator Null Safety:

- **Safe Call Operator (`?.`):** Digunakan untuk memanggil metode atau mengakses properti hanya jika objek tidak null. Jika objek null, ekspresi akan mengembalikan null.
- **Elvis Operator (`?:`):** Menyediakan nilai default jika ekspresi di sebelah kiri null.
- **Non-null Asserted Call Operator (`!!`):** Mengonversi tipe nullable menjadi non-nullable. Jika objek null, ini akan melempar `NullPointerException`. Gunakan dengan sangat hati-hati!

Contoh:

```

fun main() {
    var nama: String = "Alice" // Non-nullable
    // nama = null // Error kompilasi

    var alamat: String? = "Jalan Merdeka" // Nullable
    alamat = null
}

```

```
// Safe Call Operator
println("Panjang alamat: ${alamat?.length}") // Output: Panjang alamat: null

// Elvis Operator
val panjangAlamat = alamat?.length ?: 0
println("Panjang alamat (dengan default): $panjangAlamat") // Output: Panjang
alamat (dengan default): 0

// Non-null Asserted Call Operator (HINDARI jika tidak yakin)
// val paksaPanjang = alamat!!.length // Akan melempar NullPointerException
jika alamat null

// Smart Casts: Kotlin secara otomatis mengonversi tipe nullable ke non-
nullable setelah pemeriksaan null
if (alamat != null) {
    println("Alamat tidak null, panjangnya: ${alamat.length}")
}
}
```

1.3 Membuat Proyek Android Pertama

Sekarang setelah lingkungan pengembangan siap dan Anda memahami dasar-dasar Kotlin, mari kita buat proyek Android pertama Anda.

Langkah-langkah:

1. **Buka Android Studio.**

2. Dari layar selamat datang, pilih **New Project**.

3. Pilih **Empty Activity** dari template proyek dan klik **Next**.

4. **Konfigurasi Proyek Anda:**

- **Name:** Beri nama aplikasi Anda (misalnya, **MyFirstAndroidApp**).
- **Package name:** Ini adalah pengidentifikasi unik untuk aplikasi Anda (misalnya, **com.example.myfirstandroidapp**).
- **Save location:** Pilih lokasi untuk menyimpan proyek Anda.
- **Language:** Pilih **Kotlin**.
- **Minimum SDK version:** Pilih versi Android minimum yang ingin Anda dukung. Pilih API 21 (Android 5.0 Lollipop) atau yang lebih tinggi untuk kompatibilitas yang baik.
- Klik **Finish**.

5. **Tunggu Proses Sinkronisasi Gradle:** Android Studio akan membuat proyek dan mengunduh dependensi yang diperlukan. Proses ini mungkin memakan waktu beberapa menit tergantung pada

koneksi internet Anda.

6. Jelajahi Struktur Proyek:

Setelah proyek dimuat, Anda akan melihat struktur proyek di panel kiri. Beberapa folder dan file penting:

- `app/java/com.example.myfirstandroidapp/MainActivity.kt` : File kode Kotlin utama untuk Activity pertama Anda.
- `app/res/layout/activity_main.xml` : File layout XML yang mendefinisikan antarmuka pengguna untuk `MainActivity`.
- `app/res/mipmap/ic_launcher.xml` : Ikon aplikasi Anda.
- `app/res/values/strings.xml` : File untuk menyimpan semua string teks yang digunakan dalam aplikasi Anda, memungkinkan lokalisasi.
- `app/build.gradle` : File konfigurasi build untuk modul aplikasi Anda, tempat Anda mendeklarasikan dependensi.
- `AndroidManifest.xml` : File manifest aplikasi yang mendeklarasikan komponen aplikasi, izin, dan metadata lainnya.

7. Jalankan Aplikasi:

- **Pilih Perangkat Target:** Di toolbar atas Android Studio, pilih emulator atau perangkat fisik yang terhubung.
- **Klik Tombol Run:** Klik tombol `Run 'app'` (ikon segitiga hijau) di toolbar.
- Android Studio akan membangun aplikasi Anda dan menginstalnya di perangkat target. Anda akan melihat aplikasi "Hello World!" sederhana berjalan.

Praktikum 1.1: Aplikasi "Hello World" Kustom

Tujuan: Memodifikasi aplikasi "Hello World" default untuk menampilkan pesan kustom dan menambahkan tombol yang mengubah pesan tersebut.

Langkah-langkah:

1. **Buka** `activity_main.xml` (di `app/res/layout/`).
2. Anda akan melihat `TextView` dengan teks "Hello World!". Ubah `android:id` menjadi `"@id/myTextView"` dan `android:text` menjadi "Selamat Datang di Aplikasi Android Saya!".

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
```

```

        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:context=".MainActivity">

        <TextView
            android:id="@+id/myTextView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Selamat Datang di Aplikasi Android Saya!"
            android:textSize="24sp"
            app:layout_constraintBottom_toTopOf="@+id/myButton"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <Button
            android:id="@+id/myButton"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Ubah Pesan"
            android:layout_marginTop="16dp"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@+id/myTextView" />

    </androidx.constraintlayout.widget.ConstraintLayout>

```

3. **Tambahkan** `Button` di bawah `TextView` dengan `android:id="@+id/myButton"` dan `android:text="Ubah Pesan"`.
4. **Buka** `MainActivity.kt` (di `app/java/com.example.myfirstandroidapp/`).
5. Dapatkan referensi ke `TextView` dan `Button` menggunakan `findViewById`.
6. Tambahkan `OnClickListener` ke tombol untuk mengubah teks di `TextView`.

```

package com.example.myfirstandroidapp

import android.os.Bundle
import android.widget.Button
import android.widget.TextView

```

```
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val myTextView: TextView = findViewById(R.id.myTextView)
        val myButton: Button = findViewById(R.id.myButton)

        myButton.setOnClickListener {
            myTextView.text = "Pesan telah diubah!"
        }
    }
}
```

7. **Jalankan aplikasi** di emulator atau perangkat fisik Anda. Klik tombol "Ubah Pesan" dan amati perubahan teks.

Bagian 2: Membangun Antarmuka Pengguna (UI) Responsif

Tujuan Pembelajaran

Setelah menyelesaikan bagian ini, Anda diharapkan mampu:

- Memahami prinsip-prinsip desain UI responsif di Android.
- Menggunakan berbagai komponen `Layout` (`LinearLayout`, `RelativeLayout`, `ConstraintLayout`, `GridLayout`, `FrameLayout`) untuk mengatur tata letak elemen UI.
- Mengimplementasikan widget UI dasar seperti `TextView`, `Button`, `EditText`, dan `ImageView`.
- Memahami dan menerapkan properti UI umum seperti `padding`, `margin`, `gravity`, dan `visibility`.
- Membangun antarmuka pengguna yang adaptif dan menarik menggunakan XML.

2.1 Prinsip Desain UI Responsif

Desain UI responsif di Android berarti membuat antarmuka pengguna yang terlihat baik dan berfungsi dengan benar di berbagai ukuran layar, kepadatan piksel, dan orientasi (potret/lanskap). Dengan banyaknya perangkat Android yang tersedia, penting untuk memastikan aplikasi Anda memberikan pengalaman pengguna yang konsisten dan optimal di mana pun aplikasi tersebut dijalankan.

Konsep Kunci UI Responsif:

- **Density-independent Pixels (dp):** Gunakan `dp` (density-independent pixels) untuk menentukan ukuran dan posisi elemen UI. `dp` adalah unit abstrak yang diskalakan berdasarkan kepadatan piksel layar, memastikan ukuran elemen terlihat konsisten di berbagai perangkat.
- **Scale-independent Pixels (sp):** Gunakan `sp` (scale-independent pixels) untuk ukuran teks. `sp` mirip dengan `dp` tetapi juga diskalakan berdasarkan preferensi ukuran font pengguna.
- **Flexible Layouts:** Gunakan `ViewGroup` yang fleksibel seperti `ConstraintLayout` dan `LinearLayout` dengan `layout_weight` untuk mendistribusikan ruang secara proporsional.
- **Resource Qualifiers:** Manfaatkan direktori sumber daya alternatif (misalnya, `layout-land` untuk lanskap, `layout-sw600dp` untuk tablet) untuk menyediakan tata letak, drawable, atau nilai yang berbeda berdasarkan konfigurasi perangkat.
- **Adaptive UI Components:** Gunakan komponen UI yang secara inheren adaptif, seperti `RecyclerView` untuk daftar yang panjang dan `CardView` untuk konten yang terstruktur.

2.2 Komponen Layout Dasar (ViewGroup)

`ViewGroup` adalah wadah tak terlihat yang menampung `View` dan `ViewGroup` lainnya, serta mengatur tata letak elemen-elemen di dalamnya. Pemilihan `ViewGroup` yang tepat sangat penting untuk membangun UI yang efisien dan responsif.

2.2.1 LinearLayout

`LinearLayout` mengatur semua elemen di dalamnya dalam satu baris, baik secara horizontal maupun vertikal. Ini adalah `ViewGroup` yang paling sederhana dan sering digunakan untuk tata letak linier.

Properti Penting:

- `android:orientation` : Menentukan arah penataan elemen (`horizontal` atau `vertical`).
- `android:layout_weight` : Digunakan untuk mendistribusikan ruang yang tersedia di antara View anak secara proporsional. Bekerja paling baik dengan `layout_width="0dp"` (untuk orientasi horizontal) atau `layout_height="0dp"` (untuk orientasi vertikal).

Contoh XML (Vertikal):

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Header Aplikasi"
        android:textSize="24sp"
        android:textStyle="bold" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Tombol Aksi 1" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Tombol Aksi 2" />

</LinearLayout>
```

Contoh XML (Horizontal dengan `layout_weight`):

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="16dp">

    <Button
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Kiri" />

    <Button
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="2"
        android:text="Tengah (Lebih Besar)" />

    <Button
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Kanan" />

</LinearLayout>
```

2.2.2 RelativeLayout

RelativeLayout memungkinkan Anda memposisikan elemen UI relatif terhadap satu sama lain (misalnya, di bawah elemen lain, di sebelah kanan elemen lain) atau relatif terhadap batas **RelativeLayout** itu sendiri (misalnya, di tengah parent, di sudut kiri atas).

Properti Penting:

- **android:layout_alignParentTop**, **android:layout_alignParentBottom**, **android:layout_alignParentStart**, **android:layout_alignParentEnd** : Menempatkan View di tepi parent.
- **android:layout_centerInParent**, **android:layout_centerHorizontal**, **android:layout_centerVertical** : Menempatkan View di tengah parent.

- `android:layout_below`, `android:layout_above`, `android:layout_toStartOf`, `android:layout_toEndOf` : Menempatkan View relatif terhadap View lain (menggunakan ID View lain).
- `android:layout_alignStart`, `android:layout_alignEnd`, `android:layout_alignTop`, `android:layout_alignBottom` : Menyelaraskan tepi View dengan tepi View lain.

Contoh XML:

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/titleTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerHorizontal="true"
        android:text="Selamat Datang"
        android:textSize="28sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/usernameEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_below="@id/titleTextView"
        android:layout_marginTop="32dp"
        android:hint="Username" />

    <EditText
        android:id="@+id/passwordEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_below="@id/usernameEditText"
        android:layout_marginTop="16dp"
        android:hint="Password"
        android:inputType="textPassword" />

    <Button
        android:id="@+id/loginButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
        android:layout_below="@id/passwordEditText"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="24dp"
        android:text="Login" />

</RelativeLayout>
```

2.2.3 ConstraintLayout

ConstraintLayout adalah **ViewGroup** yang paling fleksibel dan direkomendasikan oleh Google untuk membangun tata letak UI yang kompleks dan responsif. Ini memungkinkan Anda untuk memposisikan dan mengukur View berdasarkan hubungan (constraints) antara View itu sendiri dan View lainnya, atau ke parent.

Konsep Dasar:

- **Constraints:** Setiap View harus memiliki setidaknya satu constraint horizontal dan satu constraint vertikal untuk memposisikannya. Constraint dapat diatur ke parent, ke View lain, atau ke **Guideline**.
- **Bias:** Mengatur posisi View di antara dua constraint. Misalnya, `app:layout_constraintHorizontal_bias="0.3"` akan menempatkan View 30% dari sisi kiri antara constraint kiri dan kanan.
- **Chains:** Mengatur sekelompok View yang terhubung secara linear dalam satu dimensi (horizontal atau vertikal). Ada beberapa gaya chain (spread, spread_inside, packed).
- **Guidelines:** Garis bantu yang tidak terlihat yang dapat digunakan untuk memposisikan View. Berguna untuk tata letak yang konsisten.

Contoh XML:

```
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:padding="16dp">

    <TextView
        android:id="@+id/welcomeText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Selamat Datang di Aplikasi Saya"
        android:textSize="24sp"
        android:textStyle="bold"
```

```

        app:layout_constraintBottom_toTopOf="@+id/inputField"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_chainStyle="packed" />

        <EditText
            android:id="@+id/inputField"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginTop="32dp"
            android:hint="Masukkan teks di sini"
            app:layout_constraintBottom_toTopOf="@+id/actionButton"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@+id/welcomeText" />

        <Button
            android:id="@+id/actionButton"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="24dp"
            android:text="Lakukan Aksi"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@+id/inputField" />

    </androidx.constraintlayout.widget.ConstraintLayout>

```

2.2.4 GridLayout

GridLayout mengatur elemen dalam grid persegi panjang. Anda dapat menentukan jumlah baris dan kolom, dan setiap elemen dapat menempati satu atau lebih sel.

Properti Penting:

- **android:rowCount** : Jumlah baris dalam grid.
- **android:columnCount** : Jumlah kolom dalam grid.
- **android:layout_row** , **android:layout_column** : Menentukan baris dan kolom tempat elemen akan ditempatkan.
- **android:layout_rowSpan** , **android:layout_columnSpan** : Menentukan berapa banyak baris/kolom yang akan ditempati elemen.

Contoh XML:

```
<GridLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:columnCount="3"
    android:rowCount="2"
    android:padding="16dp">

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Tombo1 1" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Tombo1 2" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Tombo1 3" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnSpan="2"
        android:layout_gravity="fill"
        android:text="Tombo1 4 (Span 2 Kolom)" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Tombo1 5" />

</GridLayout>
```

2.2.5 FrameLayout

`FrameLayout` adalah `ViewGroup` paling sederhana. Ini dirancang untuk memblokir area di layar untuk menampung satu `View`. Jika Anda menambahkan beberapa `View` ke `FrameLayout`, mereka akan ditumpuk satu di atas yang lain, dengan `View` terakhir yang ditambahkan akan berada di paling atas.

Contoh XML:

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scaleType="centerCrop"
        android:src="@drawable/background_image" /> <!-- Pastikan Anda memiliki
background_image di drawable -->

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Teks di Atas Gambar"
        android:textColor="#FFFFFF"
        android:textSize="30sp"
        android:textStyle="bold" />

</FrameLayout>
```

2.3 Widget UI Dasar

Widget adalah elemen UI interaktif yang digunakan pengguna untuk berinteraksi dengan aplikasi Anda.

2.3.1 TextView

Digunakan untuk menampilkan teks statis atau dinamis.

Properti Umum:

- `android:text`: Teks yang ditampilkan.
- `android:textSize`: Ukuran teks (sp).
- `android:textColor`: Warna teks.

- `android:textStyle` : Gaya teks (`normal` , `bold` , `italic`).
- `android:maxLines` : Jumlah maksimum baris teks.
- `android:ellipsize` : Menentukan bagaimana teks dipotong jika terlalu panjang (`start` , `middle` , `end` , `marquee`).

Contoh Kotlin:

```
val myTextView: TextView = findViewById(R.id.myTextView)
myTextView.text = "Teks baru dari Kotlin"
myTextView.setTextColor(ContextCompat.getColor(this, R.color.my_custom_color))
```

2.3.2 Button

Memungkinkan pengguna untuk memicu aksi saat diklik.

Properti Umum:

- `android:text` : Teks pada tombol.
- `android:onClick` : Nama metode di Activity yang akan dipanggil saat tombol diklik (alternatif untuk `setOnClickListener`).

Contoh Kotlin (dengan `setOnClickListener`):

```
val myButton: Button = findViewById(R.id.myButton)
myButton.setOnClickListener {
    Toast.makeText(this, "Tombol diklik!", Toast.LENGTH_SHORT).show()
}
```

2.3.3 EditText

Memungkinkan pengguna untuk memasukkan dan mengedit teks.

Properti Umum:

- `android:hint` : Teks petunjuk yang ditampilkan saat `EditText` kosong.
- `android:inputType` : Menentukan jenis input yang diharapkan (`text` , `number` , `textPassword` , `textEmailAddress` , dll.).
- `android:maxLines` : Jumlah maksimum baris yang dapat dimasukkan.

Contoh Kotlin (mengambil input):

```
val myEditText: EditText = findViewById(R.id.myEditText)
val inputText = myEditText.text.toString() // Mengambil teks
myEditText.setText("Teks yang sudah diisi") // Mengatur teks
```

2.3.4 ImageView

Digunakan untuk menampilkan gambar.

Properti Umum:

- `android:src`: Sumber gambar (dari `drawable` atau `mipmap`).
- `android:scaleType`: Menentukan bagaimana gambar diskalakan dan diposisikan di dalam `ImageView` (`centerCrop`, `fitXY`, `centerInside`, dll.).

Contoh Kotlin:

```
val myImageView: ImageView = findViewById(R.id.myImageView)
myImageView.setImageResource(R.drawable.my_image)
// Atau dari URL menggunakan library pihak ketiga seperti Glide/Picasso
// Glide.with(this).load("https://example.com/image.jpg").into(myImageView)
```

2.3.5 RecyclerView

`RecyclerView` adalah `ViewGroup` yang efisien untuk menampilkan daftar item yang besar dan dapat digulir. Ini adalah pengganti modern untuk `ListView` dan `GridView`.

Konsep Kunci:

- **Adapter**: Menghubungkan data dengan `RecyclerView` dan membuat `ViewHolder` untuk setiap item.
- **ViewHolder**: Menampung `View` untuk satu item dalam daftar.
- **LayoutManager**: Mengatur bagaimana item-item ditampilkan (misalnya, linier, grid).

Contoh Penggunaan (XML):

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/myRecyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager" />
```

Contoh Penggunaan (Kotlin - Sederhana):

```
// Di MainActivity.kt
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val recyclerView: RecyclerView = findViewById(R.id.myRecyclerView)
        val dataList = listOf("Item 1", "Item 2", "Item 3", "Item 4", "Item 5")

        // Anda perlu membuat MyAdapter dan item_layout.xml
        recyclerView.adapter = MyAdapter(dataList)
    }
}

// MyAdapter.kt
class MyAdapter(private val dataList: List<String>) :
    RecyclerView.Adapter<MyAdapter.MyViewHolder>() {

    class MyViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val textView: TextView = itemView.findViewById(R.id.itemTextView)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        MyViewHolder {
        val view =
            LayoutInflater.from(parent.context).inflate(R.layout.item_layout, parent, false)
        return MyViewHolder(view)
    }

    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
```

```
        holder.textView.text = dataList[position]
    }

    override fun getItemCount(): Int {
        return dataList.size
    }
}

// item_layout.xml (untuk setiap item di RecyclerView)
// <TextView
//     android:id="@+id/itemTextView"
//     android:layout_width="match_parent"
//     android:layout_height="wrap_content"
//     android:padding="16dp"
//     android:textSize="18sp" />
```

2.3.6 CardView

CardView adalah komponen UI yang menyediakan tampilan "kartu" dengan sudut membulat dan bayangan, memberikan efek elevasi. Ini sering digunakan untuk menampilkan konten yang terstruktur dalam blok-blok yang berbeda.

Properti Umum:

- **app:cardCornerRadius** : Radius sudut kartu.
- **app:cardElevation** : Ketinggian bayangan kartu.
- **app:cardUseCompatPadding** : Menambahkan padding tambahan pada perangkat pra-Lollipop untuk memastikan bayangan digambar dengan benar.

Contoh XML:

```
<androidx.cardview.widget.CardView
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="4dp">

    <LinearLayout
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Judul Kartu"
            android:textSize="20sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:text="Ini adalah deskripsi singkat untuk konten kartu. Anda
            bisa menempatkan gambar, teks, atau tombol di sini." />

    </LinearLayout>

</androidx.cardview.widget.CardView>

```

2.4 Properti UI Umum

Beberapa properti XML berlaku untuk hampir semua `View` dan `ViewGroup`, dan sangat penting untuk desain UI.

- `android:id`: ID unik untuk View, digunakan untuk mereferensikannya di kode Kotlin (misalnya, `R.id.myTextView`).
- `android:layout_width` dan `android:layout_height`: Menentukan lebar dan tinggi View. Nilai umum: `match_parent` (mengisi seluruh ruang yang tersedia), `wrap_content` (menyesuaikan ukuran dengan konten), atau nilai dimensi spesifik (misalnya, `100dp`).
- `android:padding`: Ruang di dalam batas View, antara konten dan batas View. Dapat diatur untuk semua sisi (`android:padding="16dp"`) atau sisi tertentu (`android:paddingStart`, `android:paddingEnd`, `android:paddingTop`, `android:paddingBottom`).
- `android:layout_margin`: Ruang di luar batas View, antara View dan View lain di sekitarnya atau batas parent. Dapat diatur untuk semua sisi (`android:layout_margin="16dp"`) atau sisi tertentu (`android:layout_marginStart`, `android:layout_marginEnd`, `android:layout_marginTop`, `android:layout_marginBottom`).

- `android:gravity` : Mengatur posisi konten di dalam View itu sendiri (misalnya, teks di dalam `TextView`). Contoh: `center` , `center_horizontal` , `center_vertical` , `left` , `right` , `top` , `bottom` .
- `android:layout_gravity` : Mengatur posisi View anak di dalam `ViewGroup` parent-nya. Ini hanya berlaku untuk `ViewGroup` tertentu seperti `LinearLayout` dan `FrameLayout` .
- `android:visibility` : Mengontrol apakah View terlihat (`visible`), tidak terlihat tetapi masih menempati ruang (`invisible`), atau tidak terlihat dan tidak menempati ruang (`gone`).

Praktikum 2.1: Mendesain Layout Profil Pengguna Responsif

Tujuan: Membangun tata letak profil pengguna yang responsif menggunakan kombinasi `ConstraintLayout` , `ImageView` , `TextView` , dan `Button` .

Langkah-langkah:

1. Buat proyek Android baru dengan `Empty Activity` .
2. Buka `activity_main.xml` . Pastikan root layout adalah `androidx.constraintlayout.widget.ConstraintLayout` .
3. Tambahkan komponen berikut:
 - Sebuah `ImageView` untuk foto profil. Anda bisa menggunakan gambar placeholder atau menambahkan gambar ke folder `drawable` .
 - Dua `TextView` untuk nama pengguna dan bio singkat.
 - Dua `Button` untuk "Edit Profil" dan "Pengaturan".

Gunakan `ConstraintLayout` untuk memposisikan elemen-elemen ini secara responsif. Pastikan elemen-elemen tetap terlihat baik saat orientasi layar berubah dari potret ke lanskap.

Tips:

- Gunakan `app:layout_constraintCircle` atau `app:layout_constraintDimensionRatio` untuk `ImageView` agar tetap proporsional.
- Gunakan `layout_margin` dan `padding` untuk jarak antar elemen.
- Pertimbangkan untuk membuat direktori `layout-land` dan membuat versi lanskap dari `activity_main.xml` jika tata letak potret tidak berfungsi dengan baik di lanskap.

Contoh `activity_main.xml` (Potret):

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
```

```
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:padding="16dp"
tools:context=".MainActivity">

<ImageView
    android:id="@+id/profileImageView"
    android:layout_width="120dp"
    android:layout_height="120dp"
    android:src="@drawable/ic_launcher_foreground" <!-- Ganti dengan
gambar profil Anda -->
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:layout_marginTop="32dp"
    android:contentDescription="Foto Profil" />

<TextView
    android:id="@+id/nameTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Nama Pengguna"
    android:textSize="24sp"
    android:textStyle="bold"
    android:layout_marginTop="16dp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/profileImageView" />

<TextView
    android:id="@+id/bioTextView"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="Seorang pengembang Android yang bersemangat belajar
dan menciptakan aplikasi inovatif."
    android:textSize="16sp"
    android:gravity="center"
    android:layout_marginTop="8dp"
    android:layout_marginStart="32dp"
    android:layout_marginEnd="32dp"
    app:layout_constraintEnd_toEndOf="parent"
```

```

        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/nameTextView" />

        <Button
            android:id="@+id/editProfileButton"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:text="Edit Profil"
            android:layout_marginTop="32dp"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@+id/bioTextView"
            app:layout_constraintWidth_percent="0.7" />

        <Button
            android:id="@+id/settingsButton"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:text="Pengaturan"
            android:layout_marginTop="16dp"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@+id/editProfileButton"
            app:layout_constraintWidth_percent="0.7" />

    </androidx.constraintlayout.widget.ConstraintLayout>

```

4. Jalankan aplikasi di emulator atau perangkat fisik. Putar orientasi layar untuk melihat bagaimana tata letak beradaptasi.

Praktikum 2.2: Membuat Daftar Item dengan RecyclerView dan CardView

Tujuan: Membangun daftar item yang dapat digulir menggunakan `RecyclerView` dan menampilkan setiap item dalam `CardView`.

Langkah-langkah:

1. Buat proyek Android baru dengan `Empty Activity`.
2. Tambahkan dependensi `RecyclerView` dan `CardView` di `build.gradle (Module: app)`:

```
dependencies {
```

```
// ... dependensi lainnya
implementation 'androidx.recyclerview:recyclerview:1.2.1'
implementation 'androidx.cardview:cardview:1.0.0'
}
```

Sinkronkan proyek Gradle setelah menambahkan dependensi.

3. **Desain** `activity_main.xml` :

Tambahkan `RecyclerView` sebagai root layout atau di dalam `ConstraintLayout` .

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/itemRecyclerView"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

4. **Buat file layout untuk setiap item** `RecyclerView` (`item_card.xml`):

Gunakan `CardView` sebagai root, dan di dalamnya, tambahkan `ImageView` dan dua `TextView` untuk judul dan deskripsi item.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```

xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_margin="8dp"
app:cardCornerRadius="8dp"
app:cardElevation="4dp">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <ImageView
        android:id="@+id/itemImageView"
        android:layout_width="match_parent"
        android:layout_height="150dp"
        android:scaleType="centerCrop"
        android:src="@drawable/ic_launcher_background" <!-- Ganti dengan
gambar placeholder -->
        android:contentDescription="Gambar Item" />

    <TextView
        android:id="@+id/itemTitleTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Judul Item"
        android:textSize="18sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/itemDescriptionTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:text="Deskripsi singkat tentang item ini." />

</LinearLayout>

</androidx.cardview.widget.CardView>

```

5. **Buat** `data class` untuk merepresentasikan data item (`Item.kt`):

```
package com.example.myfirstapp

data class Item(val title: String, val description: String, val imageUrl:
Int) // imageUrl bisa berupa Int (drawable resource) atau String (URL)
```

6. **Buat** Adapter untuk RecyclerView (ItemAdapter.kt):

```
package com.example.myfirstapp

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ImageView
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class ItemAdapter(private val itemList: List<Item>) :
RecyclerView.Adapter<ItemAdapter.ItemViewHolder>() {

    class ItemViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView)
    {
        val imageView: ImageView = itemView.findViewById(R.id.itemImageView)
        val titleTextView: TextView =
itemView.findViewById(R.id.itemTitleTextView)
        val descriptionTextView: TextView =
itemView.findViewById(R.id.itemDescriptionTextView)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
ItemViewHolder {
        val view =
LayoutInflater.from(parent.context).inflate(R.layout.item_card, parent,
false)
        return ItemViewHolder(view)
    }

    override fun onBindViewHolder(holder: ItemViewHolder, position: Int) {
        val currentItem = itemList[position]
```

```

        holder.imageView.setImageResource(currentItem.imageUrl)
        holder.titleTextView.text = currentItem.title
        holder.descriptionTextView.text = currentItem.description
    }

    override fun getItemCount(): Int {
        return itemList.size
    }
}

```

7. Inisialisasi RecyclerView di MainActivity.kt :

```

package com.example.myfirstapp

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val itemRecyclerView: RecyclerView =
            findViewById(R.id.itemRecyclerView)
        itemRecyclerView.layoutManager = LinearLayoutManager(this)

        val data = listOf(
            Item("Judul Item 1", "Deskripsi untuk item pertama.",
                R.drawable.ic_launcher_background),
            Item("Judul Item 2", "Deskripsi untuk item kedua yang sedikit
                lebih panjang.", R.drawable.ic_launcher_background),
            Item("Judul Item 3", "Deskripsi untuk item ketiga.",
                R.drawable.ic_launcher_background),
            Item("Judul Item 4", "Deskripsi untuk item keempat.",
                R.drawable.ic_launcher_background),
            Item("Judul Item 5", "Deskripsi untuk item kelima.",
                R.drawable.ic_launcher_background)
        )
    }
}

```

```
        val adapter = ItemAdapter(data)
        itemRecyclerView.adapter = adapter
    }
}
```

8. Jalankan aplikasi dan amati daftar item yang dapat digulir dengan tampilan kartu yang menarik.

Ini adalah fondasi untuk membangun UI yang responsif dan menarik di Android. Dengan menguasai berbagai `ViewGroup` dan widget, Anda dapat menciptakan antarmuka pengguna yang kompleks dan fungsional.

Bagian 3: Logika Bisnis dan Interaksi Pengguna

Tujuan Pembelajaran

Setelah menyelesaikan bagian ini, Anda diharapkan mampu:

- Menguasai konsep Pemrograman Berorientasi Objek (OOP) lanjutan di Kotlin.
- Menggunakan fitur-fitur Kotlin lanjutan seperti `abstract classes`, `sealed classes`, `generics`, dan `extension functions`.
- Memahami dan mengimplementasikan `coroutines` untuk pemrograman asinkron.
- Menangani event UI yang lebih kompleks menggunakan berbagai teknik.
- Menggunakan `ViewModel` dan `LiveData` untuk manajemen data UI yang efisien dan siklus hidup-aware.

3.1 Pemrograman Berorientasi Objek (OOP) Lanjutan di Kotlin

Pada bagian sebelumnya, kita telah membahas dasar-dasar OOP seperti `class`, `object`, `inheritance`, dan `interface`. Sekarang, mari kita selami beberapa konsep OOP yang lebih canggih di Kotlin yang akan sangat berguna dalam pengembangan Android.

3.1.1 Abstract Classes

`Abstract class` adalah kelas yang tidak dapat diinstansiasi secara langsung. Kelas ini dapat berisi properti dan metode abstrak (tanpa implementasi) serta properti dan metode non-abstrak. Kelas anak dari `abstract class` harus mengimplementasikan semua properti dan metode abstrak yang diwarisi, kecuali jika kelas anak tersebut juga dideklarasikan sebagai `abstract`.

`Abstract class` berguna ketika Anda ingin mendefinisikan kerangka dasar untuk kelas-kelas terkait, tetapi beberapa bagian dari kerangka tersebut harus diimplementasikan oleh kelas anak.

Contoh Abstract Class:

```
abstract class Bentuk {
    abstract val nama: String
    abstract fun hitungLuas(): Double

    fun tampilkanInfo() {
        println("Ini adalah bentuk $nama.")
    }
}

class Lingkaran(val radius: Double) : Bentuk() {
    override val nama: String = "Lingkaran"

    override fun hitungLuas(): Double {
        return Math.PI * radius * radius
    }
}

class Persegi(val sisi: Double) : Bentuk() {
    override val nama: String = "Persegi"

    override fun hitungLuas(): Double {
        return sisi * sisi
    }
}

fun main() {
    val lingkaran = Lingkaran(5.0)
    lingkaran.tampilkanInfo()
    println("Luas Lingkaran: ${lingkaran.hitungLuas()}")

    val persegi = Persegi(4.0)
    persegi.tampilkanInfo()
    println("Luas Persegi: ${persegi.hitungLuas()}")

    // val bentuk = Bentuk() // Error: Cannot create an instance of an abstract
    class
}
```

3.1.2 Sealed Classes

Sealed class digunakan untuk merepresentasikan hierarki kelas terbatas, di mana semua subkelasnya diketahui pada waktu kompilasi dan didefinisikan dalam file yang sama (atau dalam modul yang sama untuk Kotlin 1.5+). Ini sangat berguna ketika Anda memiliki tipe yang dapat memiliki beberapa subtype, tetapi Anda ingin membatasi subtype tersebut.

Sealed class sering digunakan bersama dengan **when** expression untuk memastikan semua kasus ditangani, sehingga compiler dapat memeriksa kelengkapan (exhaustiveness).

Contoh Sealed Class:

```
sealed class HasilOperasi {
    data class Sukses(val data: String) : HasilOperasi()
    data class Error(val kode: Int, val pesan: String) : HasilOperasi()
    object Loading : HasilOperasi()
}

fun prosesHasil(hasil: HasilOperasi) {
    when (hasil) {
        is HasilOperasi.Sukses -> println("Operasi berhasil: ${hasil.data}")
        is HasilOperasi.Error -> println("Operasi gagal: Kode ${hasil.kode},\nPesan: ${hasil.pesan}")
        HasilOperasi.Loading -> println("Sedang memuat data...")
    }
}

fun main() {
    prosesHasil(HasilOperasi.Loading)
    prosesHasil(HasilOperasi.Sukses("Data berhasil diambil!"))
    prosesHasil(HasilOperasi.Error(500, "Server tidak merespons"))
}
```

3.1.3 Generics

Generics memungkinkan Anda menulis kode yang dapat bekerja dengan berbagai tipe data tanpa mengorbankan keamanan tipe. Ini sangat berguna untuk membuat kelas, antarmuka, dan fungsi yang dapat digunakan kembali.

Contoh Generics:

```
class Kotak<T>(val isi: T) {
    fun tampilkanIsi() {
        println("Isi kotak adalah: $isi")
    }
}

fun <T> cetakArray(array: Array<T>) {
    for (item in array) {
        println(item)
    }
}

fun main() {
    val kotakString = Kotak("Halo Dunia")
    kotakString.tampilkanIsi()

    val kotakInt = Kotak(123)
    kotakInt.tampilkanIsi()

    val angka = arrayOf(1, 2, 3, 4, 5)
    cetakArray(angka)

    val nama = arrayOf("Alice", "Bob", "Charlie")
    cetakArray(nama)
}
```

3.1.4 Extension Functions

Extension function memungkinkan Anda menambahkan fungsi baru ke kelas yang sudah ada tanpa harus memodifikasi kode sumber kelas tersebut atau menggunakan pewarisan. Ini membuat kode lebih mudah dibaca dan lebih ekspresif.

Contoh Extension Function:

```
fun String.tambahSeru(): String {
    return this + "!!!"
}

fun List<Int>.rataRata(): Double {
```

```

    if (this.isEmpty()) return 0.0
    return this.sum().toDouble() / this.size
}

fun main() {
    val pesan = "Halo"
    println(pesan.tambahSeru()) // Output: Halo!!!

    val angka = listOf(1, 2, 3, 4, 5)
    println("Rata-rata: ${angka.rataRata()}") // Output: Rata-rata: 3.0

    val kosong = emptyList<Int>()
    println("Rata-rata kosong: ${kosong.rataRata()}") // Output: Rata-rata kosong:
0.0
}

```

3.2 Penanganan Event yang Lebih Kompleks

Selain `OnClickListener` dasar, ada berbagai cara untuk menangani interaksi pengguna dan event lainnya di Android.

3.2.1 Custom Listeners

Anda dapat membuat antarmuka listener kustom untuk komunikasi antara komponen UI atau antara Activity/Fragment dan komponen lainnya. Ini sangat berguna untuk decoupling kode.

Contoh Custom Listener:

```

// 1. Definisikan antarmuka listener
interface OnItemClickListener {
    fun onItemClick(item: String)
}

// 2. Implementasikan listener di Activity/Fragment
class MainActivity : AppCompatActivity(), OnItemClickListener {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Contoh penggunaan: Misalkan ada custom view yang membutuhkan listener
    }
}

```

```

        val customView = findViewById<MyCustomView>(R.id.myCustomView)
        customView.setItemClickListener(this)
    }

    override fun onItemClick(item: String) {
        Toast.makeText(this, "Item diklik: $item", Toast.LENGTH_SHORT).show()
    }
}

// 3. Buat Custom View yang menggunakan listener
class MyCustomView @JvmOverloads constructor(
    context: Context, attrs: AttributeSet? = null, defStyleAttr: Int = 0
) : LinearLayout(context, attrs, defStyleAttr) {

    private var itemClickListener: OnItemClickListener? = null
    private val myButton: Button

    init {
        LayoutInflater.from(context).inflate(R.layout.custom_view_layout, this,
true)
        myButton = findViewById(R.id.customButton)
        myButton.setOnClickListener {
            itemClickListener?.onItemClick("Data dari Custom View")
        }
    }

    fun setItemClickListener(listener: OnItemClickListener) {
        this.itemClickListener = listener
    }
}

// custom_view_layout.xml
// <LinearLayout ...>
//     <Button android:id="@+id/customButton" android:layout_width="wrap_content"
// android:layout_height="wrap_content" android:text="Klik di Custom View" />
// </LinearLayout>

```

3.2.2 Data Binding

Data Binding Library memungkinkan Anda mengikat komponen UI dalam layout ke sumber data di aplikasi Anda menggunakan format deklaratif, bukan secara terprogram. Ini mengurangi kode boilerplate dan meningkatkan keterbacaan.

Langkah-langkah Menggunakan Data Binding:

1. Aktifkan Data Binding di `build.gradle (Module: app)` :

```
android {  
    // ...  
    buildFeatures {  
        dataBinding true  
    }  
}
```

2. Bungkus layout XML Anda dengan tag `<layout>` :

```
<!-- activity_main.xml -->  
<layout xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools">  
  
    <data>  
        <variable  
            name="user"  
            type="com.example.myfirstapp.User" />  
    </data>  
  
    <androidx.constraintlayout.widget.ConstraintLayout  
        android:layout_width="match_parent"  
        android:layout_height="match_parent"  
        tools:context=".MainActivity">  
  
        <TextView  
            android:id="@+id/textViewName"  
            android:layout_width="wrap_content"  
            android:layout_height="wrap_content"  
            android:text="@{user.name}"  
            app:layout_constraintBottom_toBottomOf="parent"  
            app:layout_constraintEnd_toEndOf="parent"  
            app:layout_constraintStart_toStartOf="parent"  
            app:layout_constraintTop_toTopOf="parent" />  
    </androidx.constraintlayout.widget.ConstraintLayout>  
</layout>
```

```

        </androidx.constraintlayout.widget.ConstraintLayout>
    </layout>

```

3. Di Activity/Fragment, inialisasi binding:

```

// User.kt
data class User(val name: String, val email: String)

// MainActivity.kt
class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = DataBindingUtil setContentView(this,
            R.layout.activity_main)

        val user = User("Alice", "alice@example.com")
        binding.user = user
    }
}

```

3.2.3 TextWatcher

TextWatcher adalah antarmuka yang digunakan untuk memantau perubahan teks di **EditText**. Ini memiliki tiga metode:

- **beforeTextChanged()** : Dipanggil sebelum teks berubah.
- **onTextChanged()** : Dipanggil saat teks berubah.
- **afterTextChanged()** : Dipanggil setelah teks berubah.

Contoh TextWatcher:

```

val myEditText: EditText = findViewById(R.id.myEditText)
myEditText.addTextChangedListener(object : TextWatcher {
    override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int,
        after: Int) {
        // Do something before text changes
    }
}

```

```

    }

    override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count:
Int) {
        // Do something as text changes
    }

    override fun afterTextChanged(s: Editable?) {
        // Do something after text has changed
        val currentText = s.toString()
        println("Teks saat ini: $currentText")
    }
})

```

3.3 Coroutines untuk Pemrograman Asinkron

Coroutines adalah fitur Kotlin untuk pemrograman asinkron yang lebih sederhana dan lebih mudah dikelola daripada callback atau RxJava. Mereka memungkinkan Anda menulis kode asinkron yang terlihat seperti kode sekuensial.

Konsep Kunci:

- **suspend function**: Fungsi yang dapat dijeda dan dilanjutkan di lain waktu. Hanya dapat dipanggil dari coroutine lain atau **suspend** function lain.
- **CoroutineScope**: Mendefinisikan siklus hidup coroutine. Ketika scope dibatalkan, semua coroutine yang diluncurkan di dalamnya juga dibatalkan.
- **launch**: Memulai coroutine baru yang tidak memblokir thread saat ini dan mengembalikan **Job**.
- **async**: Memulai coroutine baru yang tidak memblokir thread saat ini dan mengembalikan **Deferred** (sejenis **Job** yang memiliki hasil).
- **Dispatchers**: Menentukan thread mana yang akan digunakan coroutine (**Dispatchers.Main**, **Dispatchers.IO**, **Dispatchers.Default**).

Contoh Coroutines Sederhana:

```

import kotlinx.coroutines.* // Pastikan Anda menambahkan dependensi coroutines di
build.gradle

fun main() = runBlocking {
    println("Mulai main")

    launch {

```

```
        delay(1000L) // Menjeda coroutine selama 1 detik
        println("Dari coroutine")
    }

    println("Akhir main")
}

// Output:
// Mulai main
// Akhir main
// (setelah 1 detik)
// Dari coroutine
```

Penggunaan di Android (dengan `ViewModelScope`):

```
// build.gradle (Module: app)
dependencies {
    // ...
    implementation "org.jetbrains.kotlinx:kotlinx-coroutines-core:1.6.4"
    implementation "org.jetbrains.kotlinx:kotlinx-coroutines-android:1.6.4"
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.5.1"
}
```

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

class MyViewModel : ViewModel() {

    fun fetchData() {
        viewModelScope.launch(Dispatchers.IO) {
            // Lakukan operasi jaringan atau database di thread IO
            val data = performNetworkRequest() // Fungsi suspend Anda

            withContext(Dispatchers.Main) {
```

```

        // Perbarui UI di thread utama
        updateUI(data)
    }
}

private suspend fun performNetworkRequest(): String {
    // Simulasi operasi jaringan yang memakan waktu
    kotlinx.coroutines.delay(2000L)
    return "Data dari server"
}

private fun updateUI(data: String) {
    // Logika untuk memperbarui UI
    println("Data diterima dan UI diperbarui: $data")
}

// Di Activity/Fragment
class MainActivity : AppCompatActivity() {
    private val viewModel: MyViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        findViewById<Button>(R.id.fetchDataButton).setOnClickListener {
            viewModel.fetchData()
        }
    }
}

```

3.4 ViewModel dan LiveData

ViewModel dan **LiveData** adalah bagian dari Android Jetpack yang membantu Anda membangun aplikasi yang kuat, dapat diuji, dan dapat dipelihara.

3.4.1 ViewModel

ViewModel dirancang untuk menyimpan dan mengelola data terkait UI dengan cara yang sadar siklus hidup. Ini berarti data yang disimpan di **ViewModel** akan bertahan dari perubahan konfigurasi (seperti rotasi layar) dan tidak akan dihancurkan saat Activity/Fragment dihancurkan dan dibuat ulang.

Manfaat ViewModel:

- **Data Persistence:** Data tetap ada saat perubahan konfigurasi.
- **Separation of Concerns:** Memisahkan logika bisnis dan data dari UI.
- **Testability:** Lebih mudah untuk menguji logika bisnis karena tidak terikat pada UI.

Contoh ViewModel:

```
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class CounterViewModel : ViewModel() {
    val count = MutableLiveData<Int>()

    init {
        count.value = 0 // Inisialisasi nilai awal
    }

    fun increment() {
        count.value = (count.value ?: 0) + 1
    }

    fun decrement() {
        count.value = (count.value ?: 0) - 1
    }
}
```

3.4.2 LiveData

LiveData adalah kelas pemegang data yang dapat diamati dan sadar siklus hidup. Ini berarti **LiveData** hanya akan memperbarui pengamat (observer) yang berada dalam status siklus hidup aktif (misalnya, **onResume**).

Manfaat LiveData:

- **Lifecycle-aware:** Memperbarui UI hanya ketika komponen UI aktif, mencegah kebocoran memori dan **NullPointerException**.
- **Data Observation:** UI secara otomatis diperbarui ketika data berubah.
- **No Memory Leaks:** Pengamat secara otomatis dihapus ketika siklus hidup mereka dihancurkan.

Contoh LiveData (Integrasi dengan ViewModel):

```
// Di MainActivity.kt
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    private val viewModel: CounterViewModel by viewModels()
    private lateinit var counterTextView: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        counterTextView = findViewById(R.id.counterTextView)
        val incrementButton: Button = findViewById(R.id.incrementButton)
        val decrementButton: Button = findViewById(R.id.decrementButton)

        // Mengamati perubahan pada LiveData
        viewModel.count.observe(this) { newCount ->
            counterTextView.text = "Count: $newCount"
        }

        incrementButton.setOnClickListener {
            viewModel.increment()
        }

        decrementButton.setOnClickListener {
            viewModel.decrement()
        }
    }
}
```

Praktikum 3.1: Aplikasi Penghitung Sederhana dengan ViewModel dan LiveData

Tujuan: Membuat aplikasi penghitung sederhana yang mempertahankan nilai hitungan saat rotasi layar menggunakan `ViewModel` dan memperbarui UI secara reaktif menggunakan `LiveData`.

Langkah-langkah:

1. Buat proyek Android baru dengan `Empty Activity`.
2. Tambahkan dependensi `lifecycle-viewmodel-ktx` dan `lifecycle-livedata-ktx` di `build.gradle (Module: app)`:

```
dependencies {  
    // ...  
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.5.1"  
    implementation "androidx.lifecycle:lifecycle-livedata-ktx:2.5.1"  
}
```

Sinkronkan proyek Gradle.

3. **Desain** `activity_main.xml`:

Tambahkan satu `TextView` untuk menampilkan hitungan dan dua `Button` untuk menambah dan mengurangi hitungan.

```
<?xml version="1.0" encoding="utf-8"?>  
<androidx.constraintlayout.widget.ConstraintLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context=".MainActivity">  
  
    <TextView  
        android:id="@+id/counterTextView"  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="Count: 0"  
        android:textSize="48sp"  
        app:layout_constraintBottom_toTopOf="@+id/incrementButton"  
        app:layout_constraintEnd_toEndOf="parent"  
        app:layout_constraintStart_toStartOf="parent"  
        app:layout_constraintTop_toTopOf="parent"  
        app:layout_constraintVertical_chainStyle="packed" />  
  
    <Button
```

```

        android:id="@+id/incrementButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:text="+"
        android:textSize="24sp"
        app:layout_constraintBottom_toTopOf="@+id/decrementButton"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/counterTextView" />

<Button
    android:id="@+id/decrementButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="-"
    android:textSize="24sp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/incrementButton" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

4. Buat CounterViewModel.kt :

```

package com.example.myfirstapp

import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class CounterViewModel : ViewModel() {
    val count = MutableLiveData<Int>()

    init {
        count.value = 0
    }

    fun increment() {
        count.value = (count.value ?: 0) + 1
    }
}

```

```
    }

    fun decrement() {
        count.value = (count.value ?: 0) - 1
    }
}
```

5. Modifikasi MainActivity.kt :

```
package com.example.myfirstapp

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    private val viewModel: CounterViewModel by viewModels()
    private lateinit var counterTextView: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        counterTextView = findViewById(R.id.counterTextView)
        val incrementButton: Button = findViewById(R.id.incrementButton)
        val decrementButton: Button = findViewById(R.id.decrementButton)

        viewModel.count.observe(this) { newCount ->
            counterTextView.text = "Count: $newCount"
        }

        incrementButton.setOnClickListener {
            viewModel.increment()
        }

        decrementButton.setOnClickListener {
            viewModel.decrement()
        }
    }
}
```

```
}  
}
```

6. Jalankan aplikasi. Klik tombol `+` dan `-`. Kemudian, putar layar perangkat Anda. Amati bahwa nilai hitungan tidak akan reset, karena disimpan dan dikelola oleh `ViewModel`.

Praktikum 3.2: Menggunakan Coroutines untuk Simulasi Operasi Jaringan

Tujuan: Mensimulasikan operasi jaringan yang memakan waktu menggunakan `coroutines` dan memperbarui UI setelah data diterima, tanpa memblokir thread utama.

Langkah-langkah:

1. Gunakan proyek dari Praktikum 3.1 atau buat proyek baru.
2. Tambahkan dependensi `kotlinx-coroutines-android` di `build.gradle (Module: app)` :

```
dependencies {  
    // ...  
    implementation "org.jetbrains.kotlin:kotlinx-coroutines-android:1.6.4"  
}
```

Sinkronkan proyek Gradle.

3. **Modifikasi** `activity_main.xml` :

Tambahkan `Button` baru (id: `fetchDataButton`, text: "Ambil Data") dan `TextView` (id: `dataResultTextView`, text: "") untuk menampilkan hasil data.

```
<?xml version="1.0" encoding="utf-8"?>  
<androidx.constraintlayout.widget.ConstraintLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context=".MainActivity">  
  
    <!-- Komponen dari Praktikum 3.1 (opsional, bisa dihapus jika fokus ke  
    coroutines) -->
```

```
<TextView
    android:id="@+id/counterTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Count: 0"
    android:textSize="48sp"
    app:layout_constraintBottom_toTopOf="@+id/incrementButton"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_chainStyle="packed" />

<Button
    android:id="@+id/incrementButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="32dp"
    android:text="+"
    android:textSize="24sp"
    app:layout_constraintBottom_toTopOf="@+id/decrementButton"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/counterTextView" />

<Button
    android:id="@+id/decrementButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="-"
    android:textSize="24sp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/incrementButton" />

<!-- Komponen baru untuk Praktikum 3.2 -->
<Button
    android:id="@+id/fetchDataButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Ambil Data"
    android:layout_marginTop="32dp"
    app:layout_constraintEnd_toEndOf="parent"
```

```

        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/decrementButton" />

        <TextView
            android:id="@+id/dataResultTextView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text=""
            android:textSize="18sp"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@+id/fetchDataButton" />

    </androidx.constraintlayout.widget.ConstraintLayout>

```

4. Modifikasi `MyViewModel.kt` (atau buat baru jika Anda membuat proyek terpisah):

Tambahkan fungsi `fetchData` yang menggunakan `viewModelScope` dan `Dispatchers.IO` untuk simulasi operasi jaringan, lalu perbarui `LiveData` di `Dispatchers.Main`.

```

package com.example.myfirstapp

import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

class MyViewModel : ViewModel() {
    val count = MutableLiveData<Int>() // Dari Praktikum 3.1
    val dataResult = MutableLiveData<String>() // LiveData baru untuk hasil
    data

    init {
        count.value = 0
        dataResult.value = "Belum ada data"
    }
}

```

```

    fun increment() {
        count.value = (count.value ?: 0) + 1
    }

    fun decrement() {
        count.value = (count.value ?: 0) - 1
    }

    fun fetchData() {
        dataResult.value = "Mengambil data..."
        viewModelScope.launch(Dispatchers.IO) {
            // Simulasi operasi jaringan yang memakan waktu
            delay(3000L) // Menjeda selama 3 detik
            val fetchedData = "Data berhasil diambil pada
            ${System.currentTimeMillis()}"

            withContext(Dispatchers.Main) {
                // Perbarui LiveData di thread utama
                dataResult.value = fetchedData
            }
        }
    }
}

```

5. Modifikasi MainActivity.kt :

Dapatkan referensi ke `fetchDataButton` dan `dataResultTextView`. Amati `dataResult` dari `ViewModel` dan panggil `fetchData()` saat tombol diklik.

```

package com.example.myfirstapp

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    private val viewModel: MyViewModel by viewModels()
    private lateinit var counterTextView: TextView // Dari Praktikum 3.1
}

```

```
private lateinit var fetchDataButton: Button
private lateinit var dataResultTextView: TextView

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    counterTextView = findViewById(R.id.counterTextView)
    val incrementButton: Button = findViewById(R.id.incrementButton)
    val decrementButton: Button = findViewById(R.id.decrementButton)
    fetchDataButton = findViewById(R.id.fetchDataButton)
    dataResultTextView = findViewById(R.id.dataResultTextView)

    // Observasi LiveData dari Praktikum 3.1
    viewModel.count.observe(this) { newCount ->
        counterTextView.text = "Count: $newCount"
    }

    incrementButton.setOnClickListener {
        viewModel.increment()
    }

    decrementButton.setOnClickListener {
        viewModel.decrement()
    }

    // Observasi LiveData untuk hasil data
    viewModel.dataResult.observe(this) { result ->
        dataResultTextView.text = result
    }

    fetchDataButton.setOnClickListener {
        viewModel.fetchData()
    }
}
```

6. Jalankan aplikasi. Klik tombol "Ambil Data". Amati bahwa UI tetap responsif (Anda masih bisa mengklik tombol **+** dan **-**) meskipun ada operasi yang memakan waktu di latar belakang. Setelah 3 detik, **dataResultTextView** akan diperbarui dengan data yang diambil.

Ini menunjukkan bagaimana **coroutines** dapat digunakan untuk melakukan operasi asinkron tanpa memblokir thread UI, dan bagaimana **ViewModel** serta **LiveData** membantu mengelola data dan

memperbarui UI dengan aman dan efisien.

Bagian 4: Navigasi dan Struktur Aplikasi

Tujuan Pembelajaran

Setelah menyelesaikan bagian ini, Anda diharapkan mampu:

- Memahami siklus hidup (Lifecycle) Activity secara mendalam dan bagaimana mengelolanya.
- Menggunakan `Explicit Intent` untuk memulai `Activity` tertentu dan mengirim data.
- Menggunakan `Implicit Intent` untuk meminta tindakan dari sistem atau aplikasi lain.
- Memahami peran `AndroidManifest.xml` dalam deklarasi komponen dan izin.
- Mengimplementasikan `Fragment` untuk membangun UI yang modular dan dapat digunakan kembali.
- Mengelola `Fragment` menggunakan `FragmentManager`.
- Menggunakan Navigation Component untuk mengelola navigasi yang kompleks dan terstruktur.

4.1 Activity Lifecycle Mendalam

Activity adalah komponen fundamental dalam aplikasi Android yang menyediakan layar tunggal dengan antarmuka pengguna. Setiap Activity memiliki siklus hidup yang dikelola oleh sistem Android, yang direpresentasikan oleh serangkaian metode callback. Memahami siklus hidup ini sangat penting untuk mengelola sumber daya aplikasi, mempertahankan status UI, dan memberikan pengalaman pengguna yang lancar.

Metode Callback Utama Activity Lifecycle:

- `onCreate(savedInstanceState: Bundle?)` : Dipanggil saat Activity pertama kali dibuat. Ini adalah tempat untuk melakukan inisialisasi satu kali yang harus terjadi hanya sekali selama masa pakai Activity, seperti mengikat data ke daftar, menginisialisasi `ViewModel`, atau mengatur tata letak (layout) menggunakan `setContentView()`. Parameter `savedInstanceState` berisi data status Activity yang disimpan sebelumnya jika Activity dibangun ulang (misalnya, setelah rotasi layar).
- `onStart()` : Dipanggil saat Activity menjadi terlihat oleh pengguna. Ini adalah tempat yang baik untuk menginisialisasi kode yang perlu berjalan saat Activity terlihat tetapi belum interaktif (misalnya, mendaftarkan `BroadcastReceiver`).
- `onResume()` : Dipanggil saat Activity mulai berinteraksi dengan pengguna. Activity berada dalam status "resumed" di bagian atas tumpukan aktivitas dan menerima input pengguna. Ini adalah tempat terbaik untuk memulai animasi, mengakses kamera, atau memulai pembaruan UI yang intensif. Metode ini dipanggil setiap kali Activity kembali ke foreground.
- `onPause()` : Dipanggil saat sistem akan melanjutkan Activity lain. Metode ini biasanya digunakan untuk menyimpan data yang belum disimpan, menghentikan animasi, atau melepaskan sumber daya yang tidak diperlukan saat Activity tidak lagi menjadi fokus utama. Ini adalah metode yang sangat cepat, jadi hindari operasi yang memakan waktu lama.

- `onStop()` : Dipanggil saat Activity tidak lagi terlihat oleh pengguna (misalnya, pengguna beralih ke aplikasi lain, atau Activity baru menutupi seluruh layar). Anda harus melepaskan sumber daya yang lebih besar yang tidak diperlukan saat Activity tidak terlihat. Activity yang dihentikan masih ada di memori.
- `onDestroy()` : Dipanggil sebelum Activity dihancurkan. Ini adalah kesempatan terakhir untuk membersihkan sumber daya yang tersisa. Ini bisa terjadi karena Activity selesai (pengguna menekan tombol kembali atau `finish()` dipanggil) atau sistem menghancurkan Activity untuk menghemat sumber daya.
- `onRestart()` : Dipanggil setelah Activity dihentikan (`onStop()`) dan akan dimulai lagi. Ini adalah tempat yang baik untuk mengembalikan status Activity yang disimpan sebelum `onStop()` dipanggil.

Diagram Activity Lifecycle:

```
graph TD
    A[App Launched] --> B{onCreate()}
    B --> C{onStart()}
    C --> D{onResume()}
    D -- "User navigates away" --> E{onPause()}
    E -- "Activity no longer visible" --> F{onStop()}
    F -- "Activity comes back to foreground" --> G{onRestart()}
    G --> C
    E -- "Activity partially visible" --> D
    F -- "Activity destroyed by system or finish() called" --> H{onDestroy()}
    D -- "User presses Back button or finish() called" --> H
```

Menyimpan dan Mengembalikan Status Activity:

Saat Activity dihancurkan oleh sistem (misalnya, karena rotasi layar atau kekurangan memori), Anda dapat menyimpan status UI menggunakan `onSaveInstanceState(outState: Bundle?)` dan mengembalikannya di `onCreate()` atau `onRestoreInstanceState(savedInstanceState: Bundle?)`.

```
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putString("my_text", myTextView.text.toString())
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

```
if (savedInstanceState != null) {  
    val savedText = savedInstanceState.getString("my_text")  
    myTextView.text = savedText  
}  
}
```

Namun, penggunaan `ViewModel` (seperti yang dibahas di Bagian 3) adalah cara yang lebih modern dan direkomendasikan untuk mempertahankan data terkait UI dari perubahan konfigurasi.

4.2 Intent: Explicit dan Implicit

`Intent` adalah objek pesan yang digunakan untuk meminta tindakan dari komponen aplikasi lain. Ini adalah mekanisme komunikasi utama antara komponen Android.

4.2.1 Explicit Intent

`Explicit Intent` digunakan ketika Anda tahu persis komponen aplikasi mana yang ingin Anda mulai. Anda menentukan nama kelas komponen target.

Kasus Penggunaan:

- Memulai `Activity` lain dalam aplikasi Anda sendiri.
- Memulai `Service` tertentu.

Contoh:

Misalkan Anda memiliki `MainActivity` dan `DetailActivity`. Anda ingin berpindah dari `MainActivity` ke `DetailActivity`.

`DetailActivity.kt` :

```
package com.example.myapp  
  
import android.os.Bundle  
import android.widget.TextView  
import androidx.appcompat.app.AppCompatActivity  
  
class DetailActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_detail)  
    }  
}
```

```

        val message = intent.getStringExtra("EXTRA_MESSAGE")
        findViewById<TextView>(R.id.detailTextView).text = message
    }
}

```

MainActivity.kt :

```

package com.example.myapp

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val editText = findViewById<EditText>(R.id.editText)
        val button = findViewById<Button>(R.id.button)

        button.setOnClickListener {
            val message = editText.text.toString()
            val intent = Intent(this, DetailActivity::class.java)
            intent.putExtra("EXTRA_MESSAGE", message) // Mengirim data
            startActivity(intent)
        }
    }
}

```

4.2.2 Implicit Intent

Implicit Intent digunakan ketika Anda ingin melakukan suatu tindakan, tetapi Anda tidak tahu atau tidak peduli komponen aplikasi mana yang akan menanganinya. Anda menentukan **action** yang ingin dilakukan dan, secara opsional, **data** yang terkait dengan **action** tersebut. Sistem Android kemudian akan mencari komponen yang terdaftar untuk menangani **Intent** tersebut.

Kasus Penggunaan:

- Membuka browser web.
- Membuat panggilan telepon.
- Mengirim email.
- Mengambil gambar dari galeri.

Contoh Implicit Intent (Membuka URL):

```
import android.content.Intent
import android.net.Uri
import android.widget.Button

// ... di dalam onCreate() Activity Anda
val openWebButton = findViewById<Button>(R.id.openWebButton)
openWebButton.setOnClickListener {
    val url = "https://www.google.com"
    val intent = Intent(Intent.ACTION_VIEW)
    intent.data = Uri.parse(url)
    // Memastikan ada aplikasi yang bisa menangani intent ini
    if (intent.resolveActivity(packageManager) != null) {
        startActivity(intent)
    }
}
```

Contoh Implicit Intent (Mengirim Email):

```
import android.content.Intent
import android.net.Uri
import android.widget.Button

// ... di dalam onCreate() Activity Anda
val sendEmailButton = findViewById<Button>(R.id.sendEmailButton)
sendEmailButton.setOnClickListener {
    val recipient = "recipient@example.com"
    val subject = "Subjek Email"
    val body = "Isi pesan email."

    val intent = Intent(Intent.ACTION_SENDTO).apply {
```

```

        data = Uri.parse("mailto:") // Hanya aplikasi email yang akan menangani
ini
        putExtra(Intent.EXTRA_EMAIL, arrayOf(recipient))
        putExtra(Intent.EXTRA_SUBJECT, subject)
        putExtra(Intent.EXTRA_TEXT, body)
    }
    if (intent.resolveActivity(packageManager) != null) {
        startActivity(intent)
    }
}

```

4.3 AndroidManifest.xml

File `AndroidManifest.xml` adalah file konfigurasi penting yang harus ada di setiap proyek Android. Ini menyediakan informasi penting tentang aplikasi Anda kepada sistem Android, yang dibutuhkan sistem sebelum dapat menjalankan kode aplikasi apa pun.

Informasi Penting dalam AndroidManifest.xml:

- **Package name:** Nama paket unik aplikasi Anda, yang berfungsi sebagai pengidentifikasi unik untuk aplikasi di perangkat dan di Google Play Store.
- **Components:** Deklarasi semua komponen aplikasi (Activity, Service, Broadcast Receiver, Content Provider). Setiap komponen harus dideklarasikan di sini agar sistem dapat menemukannya dan menjalankannya.
- **Permissions:** Izin yang dibutuhkan aplikasi untuk mengakses fitur perangkat yang dilindungi (misalnya, `android.permission.INTERNET` untuk akses internet, `android.permission.CAMERA` untuk akses kamera). Izin ini harus diminta dari pengguna saat runtime untuk izin berbahaya (dangerous permissions).
- **Hardware and software features:** Fitur hardware atau software yang dibutuhkan aplikasi (misalnya, `<uses-feature android:name="android.hardware.camera" android:required="true" />`).
- **Minimum API Level (`minSdkVersion`) dan Target API Level (`targetSdkVersion`):** Menentukan versi Android minimum yang didukung aplikasi dan versi Android yang ditargetkan untuk pengujian.
- **Application icon and label:** Ikon dan nama aplikasi yang ditampilkan di launcher perangkat.
- **Intent Filters:** Digunakan oleh `Implicit Intent` untuk mendeklarasikan kemampuan komponen aplikasi untuk merespons jenis `Intent` tertentu.

Contoh Struktur Dasar AndroidManifest.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">

```

```
<!-- Izin yang dibutuhkan aplikasi -->
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />

<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.MyApp">

    <!-- Deklarasi Activity -->
    <activity
        android:name=".MainActivity"
        android:exported="true"> <!-- exported="true" jika Activity dapat
diakses dari luar aplikasi -->
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <activity android:name=".DetailActivity" />

    <!-- Deklarasi Service (contoh) -->
    <service android:name=".MyBackgroundService" />

    <!-- Deklarasi BroadcastReceiver (contoh) -->
    <receiver android:name=".MyBroadcastReceiver">
        <intent-filter>
            <action android:name="android.intent.action.BOOT_COMPLETED" />
        </intent-filter>
    </receiver>

</application>

</manifest>
```

4.4 Fragment dan FragmentManager

Fragment adalah bagian dari antarmuka pengguna Activity yang mewakili perilaku atau bagian dari UI dalam Activity. Anda dapat menggabungkan beberapa **Fragment** dalam satu Activity untuk membangun UI multi-panel, dan Anda dapat menggunakan kembali **Fragment** di beberapa Activity.

Manfaat Fragment:

- **Modularitas:** Memungkinkan Anda memecah UI Activity menjadi komponen yang lebih kecil dan dapat dikelola.
- **Reusability:** **Fragment** dapat digunakan kembali di berbagai Activity atau dalam tata letak yang berbeda (misalnya, potret vs. lanskap).
- **Adaptability:** Memudahkan pembuatan UI yang adaptif untuk berbagai ukuran layar (misalnya, tablet).
- **Lifecycle Management:** Setiap **Fragment** memiliki siklus hidupnya sendiri, mirip dengan Activity, tetapi terkait dengan siklus hidup Activity host-nya.

4.4.1 Siklus Hidup Fragment

Siklus hidup **Fragment** mirip dengan Activity, tetapi ada beberapa metode tambahan yang terkait dengan interaksinya dengan Activity host.

Metode Callback Utama Fragment Lifecycle:

- `onAttach(context: Context)` : Dipanggil saat **Fragment** telah dikaitkan dengan Activity-nya.
- `onCreate(savedInstanceState: Bundle?)` : Dipanggil untuk melakukan inisialisasi **Fragment** (seperti `onCreate` Activity).
- `onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?)` : Dipanggil untuk menggambar UI **Fragment**. Di sinilah Anda meng-inflate layout XML **Fragment**.
- `onViewCreated(view: View, savedInstanceState: Bundle?)` : Dipanggil setelah `onCreateView()` kembali dan View **Fragment** telah dibuat. Ini adalah tempat yang baik untuk menginisialisasi View.
- `onStart()` : **Fragment** menjadi terlihat.
- `onResume()` : **Fragment** menjadi interaktif.
- `onPause()` : **Fragment** tidak lagi interaktif.
- `onStop()` : **Fragment** tidak lagi terlihat.
- `onDestroyView()` : View **Fragment** dihancurkan.
- `onDestroy()` : **Fragment** dihancurkan.
- `onDetach()` : **Fragment** tidak lagi dikaitkan dengan Activity-nya.

4.4.2 FragmentManager

FragmentManager adalah kelas yang digunakan untuk melakukan operasi pada **Fragment** dalam Activity, seperti menambah, menghapus, mengganti, dan mencari **Fragment**.

Operasi Umum FragmentManager:

- `beginTransaction()` : Memulai transaksi `Fragment` . Transaksi adalah serangkaian operasi `Fragment` yang dilakukan bersamaan.
- `add(containerId: Int, fragment: Fragment, tag: String?)` : Menambahkan `Fragment` ke `ViewGroup` dengan ID tertentu.
- `replace(containerId: Int, fragment: Fragment, tag: String?)` : Mengganti `Fragment` yang ada di `ViewGroup` dengan `Fragment` baru.
- `remove(fragment: Fragment)` : Menghapus `Fragment` .
- `addToBackStack(name: String?)` : Menambahkan transaksi ke back stack, memungkinkan pengguna untuk kembali ke status `Fragment` sebelumnya dengan tombol kembali.
- `commit()` : Melakukan transaksi `Fragment` .

Contoh Penggunaan Fragment:

`fragment_first.xml` :

```
<!-- Layout untuk FirstFragment -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Ini adalah Fragment Pertama"
        android:textSize="24sp" />

</FrameLayout>
```

`FirstFragment.kt` :

```
package com.example.myapp

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
```

```

class FirstFragment : Fragment() {
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_first, container, false)
    }
}

```

activity_main.xml (dengan `FragmentContainerView`):

```

<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragment_container"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

MainActivity.kt (Menambahkan Fragment secara dinamis):

```

package com.example.myapp

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

```

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        if (savedInstanceState == null) {  
            supportFragmentManager.beginTransaction()  
                .add(R.id.fragment_container, FirstFragment())  
                .commit()  
        }  
    }  
}
```

4.5 Navigation Component

Navigation Component adalah bagian dari Android Jetpack yang membantu Anda mengimplementasikan navigasi dalam aplikasi Android. Ini menyederhanakan implementasi navigasi, baik untuk navigasi sederhana antar layar maupun pola navigasi yang lebih kompleks seperti drawer navigasi dan bottom navigation.

Konsep Kunci:

- **Navigation Graph:** Sumber daya XML yang berisi semua tujuan navigasi (Activity, Fragment, custom views) dan tindakan (actions) yang menghubungkan tujuan tersebut.
- **NavHost:** Kontainer kosong di layout Anda yang menampilkan tujuan dari grafik navigasi Anda. Biasanya `NavHostFragment`.
- **NavController:** Objek yang mengelola navigasi aplikasi dalam `NavHost`. Ini bertanggung jawab untuk menukar konten tujuan di `NavHost`.

Manfaat Navigation Component:

- **Visualisasi Navigasi:** Memungkinkan Anda melihat alur navigasi aplikasi secara visual.
- **Penanganan Back Stack:** Mengelola back stack secara otomatis.
- **Type Safety:** Menggunakan Safe Args Gradle plugin untuk menghasilkan kode yang aman tipe untuk argumen dan navigasi.
- **Deep Linking:** Mendukung deep linking secara otomatis.

Langkah-langkah Menggunakan Navigation Component:

1. Tambahkan Dependensi:

```
// build.gradle (Module: app)
```

```
dependencies {
// ...
implementation "androidx.navigation:navigation-fragment-ktx:2.5.3"
implementation "androidx.navigation:navigation-ui-ktx:2.5.3"
}
...
```

2. Buat Navigation Graph:

Klik kanan pada folder **res** -> **New** -> **Android Resource File**. Pilih **Resource type** sebagai **Navigation** dan beri nama (misalnya, **nav_graph.xml**).

```
**Contoh `nav_graph.xml`:**
```xml

<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto"
 xmlns:tools="http://schemas.android.com/tools"
 android:id="@+id/nav_graph"
 app:startDestination="@id/firstFragment">

 <fragment
 android:id="@+id/firstFragment"
 android:name="com.example.myapp.FirstFragment"
 android:label="First Fragment"
 tools:layout="@layout/fragment_first">
 <action
 android:id="@+id/action_firstFragment_to_secondFragment"
 app:destination="@id/secondFragment" />
 </fragment>
 <fragment
 android:id="@+id/secondFragment"
 android:name="com.example.myapp.SecondFragment"
 android:label="Second Fragment"
 tools:layout="@layout/fragment_second">
 <argument
 android:name="myArg"
 app:argType="string" />
 </fragment>
</navigation>
...

```

### 3. Tambahkan `NavHostFragment` ke `activity_main.xml` :

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
 xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 tools:context=".MainActivity">

 <androidx.fragment.app.FragmentContainerView
 android:id="@+id/nav_host_fragment"
 android:name="androidx.navigation.fragment.NavHostFragment"
 android:layout_width="0dp"
 android:layout_height="0dp"
 app:layout_constraintBottom_toBottomOf="parent"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toTopOf="parent"
 app:defaultNavHost="true"
 app:navGraph="@navigation/nav_graph" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

### 4. Navigasi antar Fragment di Kotlin:

```
// Di FirstFragment.kt
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController

class FirstFragment : Fragment() {
 override fun onCreateView(
```

```
 inflater: LayoutInflater,
 container: ViewGroup?,
 savedInstanceState: Bundle?
): View? {
 val view = inflater.inflate(R.layout.fragment_first, container,
false)
 val button = view.findViewById<Button>(R.id.navigateToSecondButton)
 button.setOnClickListener {
 val action =
FirstFragmentDirections.actionFirstFragmentToSecondFragment("Pesan dari
Fragment Pertama")
 findNavController().navigate(action)
 }
 return view
 }
}

// Di SecondFragment.kt
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.navArgs

class SecondFragment : Fragment() {
 private val args: SecondFragmentArgs by navArgs()

 override fun onCreateView(
 inflater: LayoutInflater,
 container: ViewGroup?,
 savedInstanceState: Bundle?
): View? {
 val view = inflater.inflate(R.layout.fragment_second, container,
false)
 val message = args.myArg
 view.findViewById<TextView>(R.id.secondFragmentTextView).text =
message
 return view
 }
}
```

## Praktikum 4.1: Aplikasi Multi-Layar dengan Intent dan Fragment

**Tujuan:** Membangun aplikasi dengan tiga layar: layar utama (MainActivity), layar detail (DetailActivity), dan layar pengaturan (SettingsFragment yang dimuat di MainActivity).

**Langkah-langkah:**

1. Buat proyek Android baru dengan `Empty Activity`.
2. **Buat `DetailActivity` :**
  - Buat `Empty Activity` baru bernama `DetailActivity`.
  - Di `activity_detail.xml`, tambahkan `TextView` untuk menampilkan pesan yang diterima dari `MainActivity`.
  - Di `DetailActivity.kt`, ambil pesan dari `Intent` dan tampilkan di `TextView`.
  - Deklarasikan `DetailActivity` di `AndroidManifest.xml`.
3. **Buat `SettingsFragment` :**
  - Buat `Fragment (Blank)` baru bernama `SettingsFragment`.
  - Di `fragment_settings.xml`, tambahkan beberapa `TextView` atau `Switch` sederhana untuk representasi pengaturan.
  - Di `SettingsFragment.kt`, biarkan kosong untuk saat ini.
4. **Modifikasi `MainActivity` :**
  - Di `activity_main.xml`, tambahkan:
    - `Button` (id: `buttonToDetail`) untuk berpindah ke `DetailActivity`.
    - `Button` (id: `buttonToSettings`) untuk menampilkan `SettingsFragment`.
    - `FragmentContainerView` (id: `fragment_container`) untuk memuat `SettingsFragment`.
  - Di `MainActivity.kt` :
    - Tangani klik `buttonToDetail` untuk memulai `DetailActivity` menggunakan `Explicit Intent` dan kirimkan pesan.
    - Tangani klik `buttonToSettings` untuk memuat `SettingsFragment` ke dalam `fragment_container` menggunakan `FragmentManager`.

**Contoh `activity_main.xml` :**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
 xmlns:android="http://schemas.android.com/apk/res/android"
```

```

xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">

<Button
 android:id="@+id/buttonToDetail"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Pergi ke Detail"
 android:layout_marginTop="32dp"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toTopOf="parent" />

<Button
 android:id="@+id/buttonToSettings"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Buka Pengaturan"
 android:layout_marginTop="16dp"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toBottomOf="@+id/buttonToDetail" />

<androidx.fragment.app.FragmentContainerView
 android:id="@+id/fragment_container"
 android:layout_width="0dp"
 android:layout_height="0dp"
 android:layout_marginTop="32dp"
 app:layout_constraintBottom_toBottomOf="parent"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toBottomOf="@+id/buttonToSettings" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

**Contoh** MainActivity.kt :

```
package com.example.myapp
```

```

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)

 findViewById<Button>(R.id.buttonToDetail).setOnClickListener {
 val intent = Intent(this, DetailActivity::class.java)
 intent.putExtra("MESSAGE_FROM_MAIN", "Halo dari MainActivity!")
 startActivity(intent)
 }

 findViewById<Button>(R.id.buttonToSettings).setOnClickListener {
 // Memuat SettingsFragment ke dalam FragmentContainerView
 supportFragmentManager.beginTransaction()
 .replace(R.id.fragment_container, SettingsFragment())
 .addToBackStack(null) // Opsional: agar bisa kembali dengan
 tombol back
 .commit()
 }
 }
}

```

5. Jalankan aplikasi dan uji navigasi antar Activity dan pemuatan Fragment.

## Praktikum 4.2: Navigasi dengan Navigation Component

**Tujuan:** Mengimplementasikan navigasi antar dua Fragment menggunakan Navigation Component.

### Langkah-langkah:

1. Buat proyek Android baru dengan `Empty Activity`.
2. Tambahkan dependensi Navigation Component di `build.gradle (Module: app)`:

```

dependencies {
 implementation "androidx.navigation:navigation-fragment-ktx:2.5.3"
}

```

```
implementation "androidx.navigation:navigation-ui-ktx:2.5.3"
}
```

Sinkronkan proyek Gradle.

3. **Buat dua Fragment:** `HomeFragment` dan `DashboardFragment` (gunakan `Fragment (Blank)` ).

- Di `fragment_home.xml` , tambahkan `TextView` dan `Button` untuk navigasi ke `DashboardFragment` .
- Di `fragment_dashboard.xml` , tambahkan `TextView` untuk menampilkan pesan yang diterima dari `HomeFragment` .

4. **Buat Navigation Graph ( `nav_graph.xml` ):**

- Klik kanan pada folder `res` -> `New` -> `Android Resource File` . Pilih `Resource type` sebagai `Navigation` dan beri nama `nav_graph` .
- Tambahkan `HomeFragment` dan `DashboardFragment` sebagai tujuan.
- Buat `action` dari `HomeFragment` ke `DashboardFragment` .
- Tambahkan `argument` ke `DashboardFragment` untuk menerima data (misalnya, `myArg` dengan `app:argType="string"` ).

**Contoh** `nav_graph.xml` :

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto"
 xmlns:tools="http://schemas.android.com/tools"
 android:id="@+id/nav_graph"
 app:startDestination="@id/homeFragment">

 <fragment
 android:id="@+id/homeFragment"
 android:name="com.example.myapp.HomeFragment"
 android:label="Home"
 tools:layout="@layout/fragment_home">
 <action
 android:id="@+id/action_homeFragment_to_dashboardFragment"
 app:destination="@id/dashboardFragment" />
 </fragment>
 <fragment
 android:id="@+id/dashboardFragment"
```

```

 android:name="com.example.myapp.DashboardFragment"
 android:label="Dashboard"
 tools:layout="@layout/fragment_dashboard">
 <argument
 android:name="message"
 app:argType="string"
 android:defaultValue="Pesan Default" />
 </fragment>
</navigation>

```

5. Tambahkan `NavHostFragment` ke `activity_main.xml` :

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
 xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 tools:context=".MainActivity">

 <androidx.fragment.app.FragmentContainerView
 android:id="@+id/nav_host_fragment"
 android:name="androidx.navigation.fragment.NavHostFragment"
 android:layout_width="0dp"
 android:layout_height="0dp"
 app:layout_constraintBottom_toBottomOf="parent"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toTopOf="parent"
 app:defaultNavHost="true"
 app:navGraph="@navigation/nav_graph" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

6. Implementasikan Navigasi di `HomeFragment.kt` :

```
package com.example.myapp

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController

class HomeFragment : Fragment() {
 override fun onCreateView(
 inflater: LayoutInflater,
 container: ViewGroup?,
 savedInstanceState: Bundle?
): View? {
 val view = inflater.inflate(R.layout.fragment_home, container,
false)
 val button = view.findViewById<Button>(R.id.buttonToDashboard)
 button.setOnClickListener {
 val messageToSend = "Halo dari HomeFragment!"
 val action =
 HomeFragmentDirections.actionHomeFragmentToDashboardFragment(messageToSend)
 findNavController().navigate(action)
 }
 return view
 }
}
```

#### 7. Implementasikan Penerimaan Argumen di DashboardFragment.kt :

```
package com.example.myapp

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
```

```
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.navArgs

class DashboardFragment : Fragment() {
 private val args: DashboardFragmentArgs by navArgs()

 override fun onCreateView(
 inflater: LayoutInflater,
 container: ViewGroup?,
 savedInstanceState: Bundle?
): View? {
 val view = inflater.inflate(R.layout.fragment_dashboard, container,
false)
 val receivedMessage = args.message
 view.findViewById<TextView>(R.id.dashboardTextView).text =
receivedMessage
 return view
 }
}
```

8. Jalankan aplikasi dan uji navigasi antar Fragment menggunakan tombol dan tombol kembali sistem.

Bagian ini telah membahas secara komprehensif tentang bagaimana mengelola navigasi dan struktur aplikasi Android menggunakan Activity, Intent, Fragment, dan Navigation Component. Penguasaan konsep-konsep ini sangat penting untuk membangun aplikasi yang terorganisir, modular, dan mudah dinavigasi.

## Bagian 5: Penyimpanan Data Lokal (Room Database)

### Tujuan Pembelajaran

Setelah menyelesaikan bagian ini, Anda diharapkan mampu:

- Memahami konsep dasar penyimpanan data lokal di Android.
- Menggunakan Room Persistence Library untuk mengelola database SQLite.
- Mendefinisikan **Entity** untuk merepresentasikan tabel database.
- Membuat **DAO** (Data Access Object) untuk mendefinisikan metode interaksi database.
- Menginisialisasi dan mengelola **RoomDatabase**.
- Melakukan operasi CRUD (Create, Read, Update, Delete) pada data.
- Mengintegrasikan Room dengan **LiveData** atau **Flow** untuk pembaruan data secara reaktif.

### 5.1 Pengenalan Penyimpanan Data Lokal di Android

Dalam pengembangan aplikasi Android, seringkali diperlukan untuk menyimpan data secara lokal di perangkat pengguna. Ini bisa berupa preferensi pengguna, data aplikasi yang sering diakses, atau data yang perlu tersedia secara offline. Android menyediakan beberapa opsi penyimpanan data lokal, termasuk:

- **Shared Preferences:** Untuk menyimpan data primitif dalam pasangan kunci-nilai.
- **Internal/External Storage:** Untuk menyimpan file dalam jumlah besar.
- **SQLite Database:** Untuk menyimpan data terstruktur dalam format relasional.

Room Persistence Library adalah bagian dari Android Jetpack yang menyediakan lapisan abstraksi di atas SQLite. Room menyederhanakan penggunaan database SQLite dalam aplikasi Android dengan menyediakan pemetaan objek-relasional (ORM) dan menghilangkan banyak boilerplate code yang terkait dengan bekerja langsung dengan SQLiteOpenHelper. Room juga menyediakan keamanan waktu kompilasi untuk kueri SQL Anda.

## 5.2 Arsitektur Room Database

Room terdiri dari tiga komponen utama:

### 5.2.1 Entity

`Entity` merepresentasikan tabel dalam database. Setiap instance dari `Entity` adalah baris dalam tabel. Anda mendefinisikan `Entity` sebagai kelas data Kotlin yang dianotasi dengan `@Entity`.

#### Properti Penting Entity:

- `@Entity(tableName = "nama_tabel")`: Mendefinisikan kelas sebagai Entity dan menentukan nama tabel (opsional, defaultnya adalah nama kelas).
- `@PrimaryKey(autoGenerate = true)`: Mendefinisikan kunci utama untuk tabel.  
`autoGenerate = true` akan membuat Room secara otomatis menghasilkan ID unik.
- `@ColumnInfo(name = "nama_kolom")`: Mendefinisikan nama kolom dalam tabel (opsional, defaultnya adalah nama properti).
- `@Ignore`: Mengabaikan properti agar tidak disimpan di database.

#### Contoh Entity (`User.kt`):

```
package com.example.myapp.data

import androidx.room.ColumnInfo
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "users")
data class User(
```

```

 @PrimaryKey(autoGenerate = true) val uid: Int = 0,
 @ColumnInfo(name = "first_name") val firstName: String?,
 @ColumnInfo(name = "last_name") val lastName: String?
)

```

### 5.2.2 DAO (Data Access Object)

**DAO** adalah antarmuka atau kelas abstrak yang Anda gunakan untuk mendefinisikan metode interaksi database. Room akan menghasilkan implementasi dari **DAO** ini pada waktu kompilasi. Ini adalah tempat Anda menulis kueri SQL Anda.

#### Anotasi Penting DAO:

- **@Dao** : Mendefinisikan antarmuka/kelas sebagai DAO.
- **@Insert** : Untuk menyisipkan data.
- **@Update** : Untuk memperbarui data.
- **@Delete** : Untuk menghapus data.
- **@Query("SELECT \* FROM nama\_tabel")** : Untuk menulis kueri SQL kustom. Room akan memvalidasi kueri ini pada waktu kompilasi.

**Contoh DAO ( UserDao.kt ):**

```

package com.example.myapp.data

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow

@Dao
interface UserDao {
 @Query("SELECT * FROM users ORDER BY first_name ASC")
 fun getAll(): Flow<List<User>> // Mengembalikan Flow untuk data reaktif

 @Query("SELECT * FROM users WHERE uid IN (:userIds)")
 suspend fun loadAllByIds(userIds: IntArray): List<User>

 @Query("SELECT * FROM users WHERE first_name LIKE :first AND " +

```

```

 "last_name LIKE :last LIMIT 1")
 suspend fun findByName(first: String, last: String): User

 @Insert(onConflict = OnConflictStrategy.IGNORE)
 suspend fun insert(user: User)

 @Insert(onConflict = OnConflictStrategy.IGNORE)
 suspend fun insertAll(vararg users: User)

 @Update
 suspend fun update(user: User)

 @Delete
 suspend fun delete(user: User)
}

```

### 5.2.3 RoomDatabase

**RoomDatabase** adalah kelas abstrak yang berfungsi sebagai titik akses utama ke database yang mendasarinya. Kelas ini harus mewarisi dari **RoomDatabase** dan dianotasi dengan **@Database**.

#### Properti Penting Database:

- **@Database(entities = [User::class], version = 1, exportSchema = false)** :
  - **entities** : Array dari semua kelas **Entity** yang termasuk dalam database ini.
  - **version** : Versi database. Anda harus meningkatkan versi setiap kali Anda memodifikasi skema database.
  - **exportSchema** : Jika **true**, Room akan mengeksport skema database ke folder aset. Berguna untuk inspeksi dan migrasi.

**Contoh Database ( AppDatabase.kt ):**

```

package com.example.myapp.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [User::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {

```

```

abstract fun userDao(): UserDao

companion object {
 @Volatile
 private var INSTANCE: AppDatabase? = null

 fun getDatabase(context: Context): AppDatabase {
 return INSTANCE ?: synchronized(this) {
 val instance = Room.databaseBuilder(
 context.applicationContext,
 AppDatabase::class.java,
 "app_database"
).build()
 INSTANCE = instance
 instance
 }
 }
}

```

### 5.3 Operasi CRUD (Create, Read, Update, Delete)

Setelah mendefinisikan `Entity`, `DAO`, dan `RoomDatabase`, Anda dapat melakukan operasi CRUD pada data Anda.

#### Create (Insert):

```

// Di ViewModel atau Repository
fun insertUser(user: User) = viewModelScope.launch {
 userDao.insert(user)
}

```

#### Read (Query):

```

// Di ViewModel atau Repository
val allUsers: Flow<List<User>> = userDao.getAll()

// Di Activity/Fragment

```

```
viewModel.allUsers.observe(viewLifecycleOwner) { users ->
 // Perbarui UI dengan daftar pengguna
}
```

**Update:**

```
// Di ViewModel atau Repository
fun updateUser(user: User) = viewModelScope.launch {
 userDao.update(user)
}
```

**Delete:**

```
// Di ViewModel atau Repository
fun deleteUser(user: User) = viewModelScope.launch {
 userDao.delete(user)
}
```

## 5.4 Integrasi Room dengan LiveData atau Flow

Room mendukung pengembalian `LiveData` atau `Flow` dari kueri DAO. Ini memungkinkan Anda untuk mengamati perubahan data di database secara reaktif dan memperbarui UI secara otomatis ketika data berubah, tanpa perlu secara manual me-refresh data.

- **LiveData:** Cocok untuk kasus di mana Anda hanya perlu mengamati perubahan data dan memperbarui UI. `LiveData` sadar siklus hidup, sehingga akan mengelola pengamat secara otomatis.
- **Flow (Kotlin Coroutines Flow):** Lebih fleksibel dan kuat untuk kasus penggunaan yang lebih kompleks, seperti transformasi data, kombinasi beberapa sumber data, atau penanganan backpressure. `Flow` bekerja dengan `coroutines`.

Dalam contoh `UserDao` di atas, metode `getAll()` mengembalikan `Flow<List<User>>`, menunjukkan bagaimana Anda dapat menggunakan `Flow` untuk mendapatkan pembaruan data secara real-time.

### Praktikum 5.1: Aplikasi Daftar Pengguna dengan Room Database

**Tujuan:** Membuat aplikasi sederhana untuk mengelola daftar pengguna (nama depan, nama belakang) menggunakan Room Database, menampilkan daftar di `RecyclerView`, dan memungkinkan penambahan,

penghapusan, serta pembaruan data.

### Langkah-langkah:

1. Buat proyek Android baru dengan `Empty Activity`.

2. **Tambahkan Dependensi Room dan Coroutines:**

Di `build.gradle (Module: app)`, tambahkan dependensi berikut:

```
dependencies {
 // Room components
 implementation "androidx.room:room-runtime:2.5.2"
 kapt "androidx.room:room-compiler:2.5.2"
 implementation "androidx.room:room-ktx:2.5.2" // Untuk Coroutines Flow
 dan suspend functions

 // Lifecycle components (ViewModel, LiveData)
 implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.1"
 implementation "androidx.lifecycle:lifecycle-livedata-ktx:2.6.1"
 implementation "androidx.lifecycle:lifecycle-runtime-ktx:2.6.1"

 // Coroutines
 implementation "org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.1"
 implementation "org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.1"

 // RecyclerView dan CardView (jika belum ada)
 implementation "androidx.recyclerview:recyclerview:1.3.1"
 implementation "androidx.cardview:cardview:1.0.0"
}
```

Tambahkan `id 'kotlin-kapt'` di bagian atas file `build.gradle (Module: app)` jika belum ada:

```
plugins {
 id 'com.android.application'
 id 'org.jetbrains.kotlin.android'
 id 'kotlin-kapt' // Tambahkan ini
}
```

Sinkronkan proyek Gradle.

3. **Buat Entity ( User.kt ):**

Buat file `User.kt` di package `data` (misalnya, `com.example.myapp.data` ).

```
package com.example.myapp.data

import androidx.room.ColumnInfo
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "users")
data class User(
 @PrimaryKey(autoGenerate = true) val uid: Int = 0,
 @ColumnInfo(name = "first_name") val firstName: String?,
 @ColumnInfo(name = "last_name") val lastName: String?
)
```

4. **Buat DAO ( UserDao.kt ):**

Buat file `UserDao.kt` di package `data` .

```
package com.example.myapp.data

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow

@Dao
interface UserDao {
 @Query("SELECT * FROM users ORDER BY first_name ASC")
 fun getAll(): Flow<List<User>>

 @Insert(onConflict = OnConflictStrategy.IGNORE)
 suspend fun insert(user: User)
```

```

@Update
suspend fun update(user: User)

@Delete
suspend fun delete(user: User)
}

```

5. **Buat** RoomDatabase ( AppDatabase.kt ):

Buat file AppDatabase.kt di package data .

```

package com.example.myapp.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [User::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {
 abstract fun userDao(): UserDao

 companion object {
 @Volatile
 private var INSTANCE: AppDatabase? = null

 fun getDatabase(context: Context): AppDatabase {
 return INSTANCE ?: synchronized(this) {
 val instance = Room.databaseBuilder(
 context.applicationContext,
 AppDatabase::class.java,
 "user_database"
).build()
 INSTANCE = instance
 instance
 }
 }
 }
}

```

6. Buat `Repository` ( `UserRepository.kt` ):

Ini adalah lapisan abstraksi antara `ViewModel` dan `DAO` .

```
package com.example.myapp.data

import androidx.annotation.WorkerThread
import kotlinx.coroutines.flow.Flow

class UserRepository(private val userDao: UserDao) {

 val allUsers: Flow<List<User>> = userDao.getAll()

 @Suppress("RedundantSuspendModifier")
 @WorkerThread
 suspend fun insert(user: User) {
 userDao.insert(user)
 }

 @Suppress("RedundantSuspendModifier")
 @WorkerThread
 suspend fun update(user: User) {
 userDao.update(user)
 }

 @Suppress("RedundantSuspendModifier")
 @WorkerThread
 suspend fun delete(user: User) {
 userDao.delete(user)
 }
}
```

7. Buat `ViewModel` ( `UserViewModel.kt` ):

```
package com.example.myapp.ui

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.asLiveData
```

```

import androidx.lifecycle.viewModelScope
import com.example.myapp.data.User
import com.example.myapp.data.UserRepository
import kotlinx.coroutines.launch

class UserViewModel(private val repository: UserRepository) : ViewModel() {

 val allUsers = repository.allUsers.asLiveData()

 fun insert(user: User) = viewModelScope.launch {
 repository.insert(user)
 }

 fun update(user: User) = viewModelScope.launch {
 repository.update(user)
 }

 fun delete(user: User) = viewModelScope.launch {
 repository.delete(user)
 }
}

class UserViewModelFactory(private val repository: UserRepository) :
 ViewModelProvider.Factory {
 override fun <T : ViewModel> create(modelClass: Class<T>): T {
 if (modelClass.isAssignableFrom(UserViewModel::class.java)) {
 @Suppress("UNCHECKED_CAST")
 return UserViewModel(repository) as T
 }
 throw IllegalArgumentException("Unknown ViewModel class")
 }
}

```

#### 8. Buat Application Class ( UserApplication.kt ):

Untuk menyediakan instance database dan repository.

```

package com.example.myapp

import android.app.Application
import com.example.myapp.data.AppDatabase
import com.example.myapp.data.UserRepository

```

```
class UserApplication : Application() {
 val database by lazy { AppDatabase.getDatabase(this) }
 val repository by lazy { UserRepository(database.userDao()) }
}
```

Tambahkan nama kelas ini ke `AndroidManifest.xml` di tag `<application>` :

```
<application
 android:name=".UserApplication"
 ...
</application>
```

#### 9. Desain `activity_main.xml` :

Tambahkan `EditText` untuk nama depan dan nama belakang, `Button` untuk menambah pengguna, dan `RecyclerView` untuk menampilkan daftar pengguna.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
 xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:padding="16dp"
 tools:context=".MainActivity">

 <EditText
 android:id="@+id/editTextFirstName"
 android:layout_width="0dp"
 android:layout_height="wrap_content"
 android:hint="Nama Depan"
 android:inputType="textPersonName"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toTopOf="parent" />
```

```

<EditText
 android:id="@+id/editTextLastName"
 android:layout_width="0dp"
 android:layout_height="wrap_content"
 android:layout_marginTop="8dp"
 android:hint="Nama Belakang"
 android:inputType="textPersonName"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toBottomOf="@+id/editTextFirstName" />

<Button
 android:id="@+id/buttonAddUser"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:layout_marginTop="16dp"
 android:text="Tambah Pengguna"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toBottomOf="@+id/editTextLastName" />

<androidx.recyclerview.widget.RecyclerView
 android:id="@+id/recyclerViewUsers"
 android:layout_width="0dp"
 android:layout_height="0dp"
 android:layout_marginTop="16dp"
 app:layout_constraintBottom_toBottomOf="parent"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toBottomOf="@+id/buttonAddUser"
 tools:listitem="@layout/item_user" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

10. Buat file layout untuk setiap item `RecyclerView` ( `item_user.xml` ):

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:orientation="vertical"

```

```

 android:padding="8dp">

 <TextView
 android:id="@+id/textViewUserName"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:textSize="18sp"
 android:textStyle="bold"
 tools:text="John Doe" />

 <TextView
 android:id="@+id/textViewUserId"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:textSize="14sp"
 tools:text="ID: 1" />

 <Button
 android:id="@+id/buttonDeleteUser"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Hapus"
 android:layout_gravity="end" />

 </LinearLayout>

```

11. Buat Adapter untuk RecyclerView ( UserListAdapter.kt ):

```

package com.example.myapp.ui

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.TextView
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.example.myapp.R
import com.example.myapp.data.User

```

```

class UserListAdapter(private val onDeleteClick: (User) -> Unit) :
ListAdapter<User, UserListAdapter.UserViewHolder>(UsersComparator()) {

 override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
UserViewHolder {
 return UserViewHolder.create(parent)
 }

 override fun onBindViewHolder(holder: UserViewHolder, position: Int) {
 val current = getItem(position)
 holder.bind(current, onDeleteClick)
 }

 class UserViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView)
 {
 private val userNameTextView: TextView =
itemView.findViewById(R.id.textViewUserName)
 private val userIdTextView: TextView =
itemView.findViewById(R.id.textViewUserId)
 private val deleteButton: Button =
itemView.findViewById(R.id.buttonDeleteUser)

 fun bind(user: User, onDeleteClick: (User) -> Unit) {
 userNameTextView.text = "${user.firstName} ${user.lastName}"
 userIdTextView.text = "ID: ${user.uid}"
 deleteButton.setOnClickListener { onDeleteClick(user) }
 }

 companion object {
 fun create(parent: ViewGroup): UserViewHolder {
 val view: View = LayoutInflater.from(parent.context)
 .inflate(R.layout.item_user, parent, false)
 return UserViewHolder(view)
 }
 }
 }

 class UsersComparator : DiffUtil.ItemCallback<User>() {
 override fun areItemsTheSame(oldItem: User, newItem: User): Boolean
 {
 return oldItem === newItem
 }

 override fun areContentsTheSame(oldItem: User, newItem: User):

```

```
Boolean {
 return oldItem == newItem
}
}
}
```

## 12. Modifikasi MainActivity.kt :

```
package com.example.myapp

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.myapp.data.User
import com.example.myapp.ui.UserListAdapter
import com.example.myapp.ui.UserViewModel
import com.example.myapp.ui.UserViewModelFactory

class MainActivity : AppCompatActivity() {

 private val userViewModel: UserViewModel by viewModels {
 UserViewModelFactory((application as UserApplication).repository)
 }

 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)

 val editTextFirstName = findViewById<EditText>
(R.id.editTextFirstName)
 val editTextLastName = findViewById<EditText>(R.id.editTextLastName)
 val buttonAddUser = findViewById<Button>(R.id.buttonAddUser)
 val recyclerViewUsers = findViewById<RecyclerView>
(R.id.recyclerViewUsers)

 val adapter = UserListAdapter { user ->
 userViewModel.delete(user)
 }
 }
}
```

```

 }
 recyclerViewUsers.adapter = adapter
 recyclerViewUsers.layoutManager = LinearLayoutManager(this)

 userModel.allUsers.observe(this) { users ->
 users.let { adapter.submitList(it) }
 }

 buttonAddUser.setOnClickListener {
 val firstName = editTextFirstName.text.toString()
 val lastName = editTextLastName.text.toString()
 if (firstName.isNotEmpty() && lastName.isNotEmpty()) {
 userModel.insert(User(firstName = firstName, lastName =
lastName))

 editTextFirstName.setText("")
 editTextLastName.setText("")
 }
 }
}
}
}

```

13. Jalankan aplikasi. Anda sekarang dapat menambah dan menghapus pengguna, dan daftar akan diperbarui secara real-time berkat `LiveData` dan `Flow`.

Bagian ini telah memberikan pemahaman yang komprehensif tentang bagaimana mengimplementasikan penyimpanan data lokal menggunakan Room Database di Android. Dengan menguasai `Entity`, `DAO`, `RoomDatabase`, dan mengintegrasikannya dengan `ViewModel` dan `LiveData` / `Flow`, Anda dapat membangun aplikasi yang mampu mengelola data secara persisten dan reaktif.

## Bagian 6: Konektivitas Jaringan dan RESTful API

### Tujuan Pembelajaran

Setelah menyelesaikan bagian ini, Anda diharapkan mampu:

- Memahami konsep dasar RESTful API dan format data JSON.
- Menggunakan library Retrofit untuk melakukan permintaan HTTP ke API.
- Melakukan parsing respons JSON menjadi objek Kotlin menggunakan GSON atau `kotlinx.serialization`.
- Menampilkan data yang diambil dari API ke antarmuka pengguna (UI), khususnya menggunakan `RecyclerView`.
- Menangani berbagai skenario error jaringan dan menampilkan pesan yang sesuai kepada pengguna.

## 6.1 Pengenalan RESTful API dan JSON

Sebagian besar aplikasi modern tidak berdiri sendiri; mereka berinteraksi dengan layanan backend untuk mengambil atau mengirim data. Interaksi ini seringkali dilakukan melalui **API (Application Programming Interface)**. Salah satu arsitektur API yang paling populer adalah **REST (Representational State Transfer)**.

### 6.1.1 RESTful API

RESTful API adalah gaya arsitektur untuk sistem terdistribusi yang menyediakan standar untuk bagaimana sistem komputer dapat berkomunikasi di web. Prinsip-prinsip utama REST meliputi:

- **Client-Server:** Pemisahan kekhawatiran antara klien dan server. Klien bertanggung jawab untuk UI, server untuk data dan logika bisnis.
- **Stateless:** Setiap permintaan dari klien ke server harus berisi semua informasi yang diperlukan untuk memahami permintaan tersebut. Server tidak menyimpan konteks klien antara permintaan.
- **Cacheable:** Respons dari server dapat di-cache untuk meningkatkan kinerja.
- **Layered System:** Klien tidak dapat mengetahui apakah ia terhubung langsung ke server akhir atau ke perantara (proxy, load balancer).
- **Uniform Interface:** Ini adalah prinsip inti REST, yang meliputi:
  - **Resource Identification in Requests:** Sumber daya diidentifikasi menggunakan URI (Uniform Resource Identifier).
  - **Resource Manipulation Through Representations:** Klien memanipulasi sumber daya melalui representasi yang diterima dari server (misalnya, JSON atau XML).
  - **Self-descriptive Messages:** Setiap pesan berisi informasi yang cukup untuk menjelaskan bagaimana memproses pesan tersebut.
  - **Hypermedia as the Engine of Application State (HATEOAS):** Klien berinteraksi dengan aplikasi sepenuhnya melalui hyperlink yang disediakan oleh server.

#### Metode HTTP Umum dalam REST:

- **GET:** Mengambil data dari server.
- **POST:** Mengirim data baru ke server untuk membuat sumber daya baru.
- **PUT:** Memperbarui sumber daya yang ada secara keseluruhan.
- **PATCH:** Memperbarui sebagian sumber daya yang ada.
- **DELETE:** Menghapus sumber daya dari server.

### 6.1.2 JSON (JavaScript Object Notation)

JSON adalah format pertukaran data yang ringan, mudah dibaca manusia, dan mudah di-parse oleh mesin. Ini adalah format yang paling umum digunakan dalam RESTful API.

#### Struktur Dasar JSON:

- **Objek:** Direpresentasikan oleh kurung kurawal `{ }`. Berisi pasangan kunci-nilai. Kunci adalah string, nilai bisa berupa string, angka, boolean, null, objek lain, atau array.

```
{
 "nama": "Budi",
 "umur": 30,
 "is_active": true
}
```

- **Array:** Direpresentasikan oleh kurung siku `[]`. Berisi daftar nilai yang dipisahkan koma.

```
[
 {
 "nama": "Budi",
 "umur": 30
 },
 {
 "nama": "Ani",
 "umur": 25
 }
]
```

#### Contoh Data JSON dari API:

Misalkan Anda mengambil daftar film dari API:

```
{
 "page": 1,
 "total_pages": 500,
 "total_results": 10000,
 "results": [
 {
 "id": 1,
 "title": "Inception",
 "overview": "A thief who steals corporate secrets...",
 "release_date": "2010-07-16",
 "poster_path": "/poster1.jpg"
 },
 {
 "id": 2,
 "title": "The Matrix",
 "overview": "A computer hacker learns from mysterious rebels..."
 }
]
}
```

```
 "release_date": "1999-03-31",
 "poster_path": "/poster2.jpg"
 }
]
}
```

## 6.2 Menggunakan Retrofit untuk Permintaan HTTP

Retrofit adalah HTTP client yang aman tipe untuk Android dan Java yang dikembangkan oleh Square. Ini sangat memudahkan proses membuat permintaan HTTP ke RESTful API dan mengurai respons menjadi objek Kotlin/Java.

### Langkah-langkah Menggunakan Retrofit:

#### 1. Tambahkan Dependensi:

Di `build.gradle (Module: app)`, tambahkan dependensi Retrofit dan converter (misalnya, GSON untuk JSON parsing).

```
dependencies {
 // Retrofit
 implementation "com.squareup.retrofit2:retrofit:2.9.0"
 implementation "com.squareup.retrofit2:converter-gson:2.9.0" // Untuk
 konversi JSON ke objek Kotlin

 // OkHttp (biasanya disertakan oleh Retrofit, tapi bisa ditambahkan
 secara eksplisit jika perlu)
 implementation "com.squareup.okhttp3:okhttp:4.9.0"
 implementation "com.squareup.okhttp3:logging-interceptor:4.9.0" // Untuk
 logging request/response
}
```

Sinkronkan proyek Gradle.

#### 2. Buat Data Class (Model) untuk Respons JSON:

Berdasarkan struktur JSON yang diharapkan dari API, buat kelas data Kotlin yang sesuai. Gunakan `@SerializedName` jika nama properti JSON berbeda dengan nama properti Kotlin Anda.

```
// Movie.kt
package com.example.myapp.data.model
```

```
import com.google.gson.annotations.SerializedName

data class Movie(
 val id: Int,
 val title: String,
 val overview: String,
 @SerializedName("release_date") val releaseDate: String,
 @SerializedName("poster_path") val posterPath: String?
)

// MovieResponse.kt (jika API mengembalikan objek dengan daftar film)
data class MovieResponse(
 val page: Int,
 @SerializedName("total_pages") val totalPages: Int,
 @SerializedName("total_results") val totalResults: Int,
 val results: List<Movie>
)
```

### 3. Buat Antarmuka Service API:

Definisikan antarmuka Kotlin dengan anotasi Retrofit yang menjelaskan permintaan HTTP (GET, POST, dll.) dan endpoint API.

```
// MovieApiService.kt
package com.example.myapp.network

import com.example.myapp.data.model.MovieResponse
import retrofit2.Response
import retrofit2.http.GET
import retrofit2.http.Query

interface MovieApiService {
 @GET("movie/popular")
 suspend fun getPopularMovies(
 @Query("api_key") apiKey: String,
 @Query("language") language: String = "en-US",
 @Query("page") page: Int = 1
): Response<MovieResponse>

 // Contoh lain:
 // @POST("user/login")
```

```
// suspend fun loginUser(@Body request: LoginRequest):
Response<LoginResponse>
}
```

- `@GET`, `@POST`, `@PUT`, `@DELETE` : Menentukan metode HTTP dan path relatif ke base URL.
- `@Query` : Menambahkan parameter kueri ke URL.
- `@Path` : Mengganti bagian dari URL dengan nilai dinamis.
- `@Body` : Mengirim objek sebagai body permintaan (untuk POST/PUT).
- `suspend` : Menandakan bahwa fungsi ini adalah `suspend function` dan dapat dipanggil dari `coroutine`.
- `Response<T>` : Mengembalikan objek `Response` dari Retrofit yang berisi respons API dan informasi tambahan seperti kode status HTTP.

#### 4. Buat Instance Retrofit:

Gunakan `Retrofit.Builder` untuk mengonfigurasi Retrofit, termasuk base URL dan converter factory.

```
// RetrofitClient.kt
package com.example.myapp.network

import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitClient {
 private const val BASE_URL = "https://api.themoviedb.org/3/" // Contoh
 Base URL

 private val loggingInterceptor = HttpLoggingInterceptor().apply {
 level = HttpLoggingInterceptor.Level.BODY // Log request dan
 response body
 }

 private val okHttpClient = OkHttpClient.Builder()
 .addInterceptor(loggingInterceptor)
 .build()

 val instance: MovieApiService by lazy {
 Retrofit.Builder()
 .baseUrl(BASE_URL)
 .addConverterFactory(GsonConverterFactory.create())
 .addInterceptor(loggingInterceptor)
 .client(okHttpClient)
 .build()
 .create(MovieApiService::class.java)
 }
}
```

```

 .baseUrl(BASE_URL)
 .client(okHttpClient)
 .addConverterFactory(GsonConverterFactory.create())
 .build()
 .create(MovieApiService::class.java)
 }
}

```

##### 5. Lakukan Permintaan API (di ViewModel atau Repository):

Panggil metode dari antarmuka service API Anda di dalam `coroutine` .

```

// MovieRepository.kt
package com.example.myapp.data.repository

import com.example.myapp.data.model.Movie
import com.example.myapp.network.MovieApiService
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow
import kotlinx.coroutines.flow.flowOn

class MovieRepository(private val apiService: MovieApiService) {

 fun getPopularMovies(apiKey: String, page: Int): Flow<List<Movie>> =
 flow {
 val response = apiService.getPopularMovies(apiKey, page = page)
 if (response.isSuccessful) {
 response.body()?.results?.let { movies ->
 emit(movies)
 }
 } else {
 // Tangani error, misalnya throw exception atau emit error state
 throw Exception("Error fetching movies: ${response.code()}")
 }
 }.flowOn(Dispatchers.IO)
}

// MovieViewModel.kt
package com.example.myapp.ui.movie

import androidx.lifecycle.LiveData

```

```
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.asLiveData
import androidx.lifecycle.viewModelScope
import com.example.myapp.data.model.Movie
import com.example.myapp.data.repository.MovieRepository
import kotlinx.coroutines.launch

class MovieViewModel(private val repository: MovieRepository) : ViewModel()
{

 private val _movies = MutableLiveData<List<Movie>>()
 val movies: LiveData<List<Movie>> = _movies

 private val _errorMessage = MutableLiveData<String>()
 val errorMessage: LiveData<String> = _errorMessage

 fun fetchPopularMovies(apiKey: String, page: Int = 1) {
 viewModelScope.launch {
 try {
 repository.getPopularMovies(apiKey, page).collect {
 _movies.value = it
 }
 } catch (e: Exception) {
 _errorMessage.value = "Gagal mengambil film: ${e.message}"
 }
 }
 }
}

class MovieViewModelFactory(private val repository: MovieRepository) :
 ViewModelProvider.Factory {
 override fun <T : ViewModel> create(modelClass: Class<T>): T {
 if (modelClass.isAssignableFrom(MovieViewModel::class.java)) {
 @Suppress("UNCHECKED_CAST")
 return MovieViewModel(repository) as T
 }
 throw IllegalArgumentException("Unknown ViewModel class")
 }
}
```

### 6.3 Menampilkan Data dari API ke UI (RecyclerView)

Setelah data berhasil diambil dan di-parse menjadi objek Kotlin, langkah selanjutnya adalah menampilkannya di UI. `RecyclerView` adalah pilihan yang sangat baik untuk menampilkan daftar data yang dinamis.

### Langkah-langkah:

#### 1. Desain Item Layout untuk `RecyclerView` :

Buat file XML baru (misalnya, `item_movie.xml`) yang akan merepresentasikan tata letak setiap item film dalam daftar. Ini bisa berupa `CardView` yang berisi `ImageView` untuk poster film dan `TextView` untuk judul dan deskripsi.

```
<!-- item_movie.xml -->
<androidx.cardview.widget.CardView
 xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:layout_margin="8dp"
 app:cardCornerRadius="8dp"
 app:cardElevation="4dp">

 <LinearLayout
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:orientation="vertical"
 android:padding="16dp">

 <ImageView
 android:id="@+id/moviePosterImageView"
 android:layout_width="match_parent"
 android:layout_height="200dp"
 android:scaleType="centerCrop"
 android:src="@drawable/ic_launcher_background" /> <!--
Placeholder -->

 <TextView
 android:id="@+id/movieTitleTextView"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:layout_marginTop="8dp"
 android:textSize="18sp"
 android:textStyle="bold"
 android:text="Judul Film" />
```

```

 <TextView
 android:id="@+id/movieOverviewTextView"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:layout_marginTop="4dp"
 android:maxLines="3"
 android:ellipsize="end"
 android:text="Ringkasan film yang panjang dan menarik..." />

 </LinearLayout>

</androidx.cardview.widget.CardView>

```

## 2. Buat Adapter untuk RecyclerView :

Buat kelas adapter yang akan mengikat data `Movie` ke tampilan `item_movie.xml` .

```

// MovieAdapter.kt
package com.example.myapp.ui.movie

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ImageView
import android.widget.TextView
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.bumptech.glide.Glide // Untuk memuat gambar dari URL
import com.example.myapp.R
import com.example.myapp.data.model.Movie

class MovieAdapter : ListAdapter<Movie, MovieAdapter.MovieViewHolder>
(MovieDiffCallback()) {

 override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
 MovieViewHolder {
 val view =
 LayoutInflater.from(parent.context).inflate(R.layout.item_movie, parent,
 false)
 }

```

```

 return MovieViewHolder(view)
 }

 override fun onBindViewHolder(holder: MovieViewHolder, position: Int) {
 val movie = getItem(position)
 holder.bind(movie)
 }

 class MovieViewHolder(itemView: View) :
 RecyclerView.ViewHolder(itemView) {
 private val posterImageView: ImageView =
 itemView.findViewById(R.id.moviePosterImageView)
 private val titleTextView: TextView =
 itemView.findViewById(R.id.movieTitleTextView)
 private val overviewTextView: TextView =
 itemView.findViewById(R.id.movieOverviewTextView)

 fun bind(movie: Movie) {
 titleTextView.text = movie.title
 overviewTextView.text = movie.overview
 // Memuat gambar menggunakan Glide (pastikan Anda menambahkan
 // dependensi Glide)
 movie.posterPath?.let {
 val imageUrl = "https://image.tmdb.org/t/p/w500" + it //
 // Base URL untuk gambar TMDB
 Glide.with(itemView.context)
 .load(imageUrl)
 .placeholder(R.drawable.ic_launcher_background) //
 // Placeholder gambar
 .error(R.drawable.ic_launcher_foreground) // Gambar
 // error
 .into(posterImageView)
 }
 }
 }

 class MovieDiffCallback : DiffUtil.ItemCallback<Movie>() {
 override fun areItemsTheSame(oldItem: Movie, newItem: Movie):
 Boolean {
 return oldItem.id == newItem.id
 }

 override fun areContentsTheSame(oldItem: Movie, newItem: Movie):
 Boolean {

```

```

 return oldItem == newItem
 }
}

```

**Catatan:** Untuk memuat gambar dari URL, Anda memerlukan library pihak ketiga seperti Glide atau Picasso. Tambahkan dependensi Glide di `build.gradle (Module: app)` :

```

dependencies {
 implementation 'com.github.bumptech.glide:glide:4.15.1'
 annotationProcessor 'com.github.bumptech.glide:compiler:4.15.1'
}

```

Dan tambahkan izin internet di `AndroidManifest.xml` :

```

<uses-permission android:name="android.permission.INTERNET" />

```

### 3. Inisialisasi `RecyclerView` di `MainActivity` :

```

// MainActivity.kt
package com.example.myapp

import android.os.Bundle
import android.widget.TextView
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.myapp.network.RetrofitClient
import com.example.myapp.data.repository.MovieRepository
import com.example.myapp.ui.movie.MovieAdapter
import com.example.myapp.ui.movie.MovieViewModel
import com.example.myapp.ui.movie.MovieViewModelFactory

```

```

class MainActivity : AppCompatActivity() {

 private val movieViewModel: MovieViewModel by viewModels {
 MovieViewModelFactory(MovieRepository(RetrofitClient.instance))
 }

 private lateinit var movieAdapter: MovieAdapter
 private lateinit var recyclerView: RecyclerView
 private lateinit var errorMessageTextView: TextView

 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)

 recyclerView = findViewById(R.id.recyclerViewMovies)
 errorMessageTextView = findViewById(R.id.errorMessageTextView)

 movieAdapter = MovieAdapter()
 recyclerView.layoutManager = LinearLayoutManager(this)
 recyclerView.adapter = movieAdapter

 // Observasi LiveData untuk daftar film
 movieViewModel.movies.observe(this) { movies ->
 movies?.let { movieAdapter.submitList(it) }
 }

 // Observasi LiveData untuk pesan error
 movieViewModel.errorMessage.observe(this) { message ->
 errorMessageTextView.text = message
 errorMessageTextView.visibility = if (message.isNullOrEmpty())
View.GONE else View.VISIBLE
 }

 // Panggil API untuk mengambil film
 // Ganti

```

API\_KEY\_ANDA\_DI\_SINI" dengan kunci API TMDB Anda yang sebenarnya

movieViewModel.fetchPopularMovies("API\_KEY\_ANDA\_DI\_SINI")

}

}

...

## 6.4 Penanganan Error Jaringan

Penanganan error adalah bagian krusial dari setiap aplikasi yang berinteraksi dengan jaringan. Aplikasi Anda harus mampu menangani berbagai skenario kegagalan, seperti:

- Tidak ada koneksi internet.
- Server tidak merespons atau error server (kode status HTTP 5xx).
- Permintaan tidak valid (kode status HTTP 4xx).
- Waktu habis (timeout).
- Data yang diterima tidak sesuai format.

### Strategi Penanganan Error:

- **Try-Catch Blocks:** Gunakan `try-catch` di sekitar panggilan API asinkron (misalnya, di dalam `coroutine`) untuk menangkap `IOException` (masalah jaringan) atau `HttpException` (respons HTTP non-2xx).
- **Respons Retrofit:** Objek `Response<T>` dari Retrofit memiliki metode `isSuccessful()` untuk memeriksa apakah respons berada dalam rentang 200-300. Jika tidak, Anda bisa mendapatkan kode error dan pesan error dari `response.code()` dan `response.errorBody()`.
- **UI Feedback:** Berikan umpan balik yang jelas kepada pengguna saat terjadi error (misalnya, `Toast`, `Snackbar`, `TextView` error, atau dialog).
- **Retry Mechanism:** Pertimbangkan untuk mengimplementasikan mekanisme coba lagi (retry) untuk error jaringan sementara.

### Contoh Penanganan Error di ViewModel (sudah termasuk di `MovieViewModel` sebelumnya):

```
// ... di MovieViewModel.kt
fun fetchPopularMovies(apiKey: String, page: Int = 1) {
 viewModelScope.launch {
 try {
 repository.getPopularMovies(apiKey, page).collect {
 _movies.value = it
 _errorMessage.value = null // Hapus pesan error jika berhasil
 }
 } catch (e: Exception) {
 // Tangani berbagai jenis exception
 _errorMessage.value = when (e) {
 is java.io.IOException -> "Periksa koneksi internet Anda."
 is retrofit2.HttpException -> {
 val errorBody = e.response()?.errorBody()?.string()
 "Error API: ${e.code()} - ${errorBody ?: e.message()}"
 }
 }
 }
 }
}
```

```

 else -> "Terjadi kesalahan tidak terduga: ${e.message}"
 }
}
}
}

```

## Praktikum 6.1: Mengambil dan Menampilkan Daftar Film dari TMDB API

**Tujuan:** Membangun aplikasi yang mengambil daftar film populer dari The Movie Database (TMDB) API dan menampilkannya dalam `RecyclerView`.

### Langkah-langkah:

#### 1. Dapatkan Kunci API TMDB:

- Kunjungi <https://www.themoviedb.org/> dan daftar/masuk.
- Pergi ke pengaturan akun Anda -> API -> Request an API Key (Developer).
- Simpan kunci API Anda, Anda akan membutuhkannya.

#### 2. Buat proyek Android baru dengan `Empty Activity`.

#### 3. Tambahkan Izin Internet di `AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.INTERNET" />
```

#### 4. Tambahkan Dependensi di `build.gradle (Module: app)`:

```

dependencies {
 // Retrofit
 implementation "com.squareup.retrofit2:retrofit:2.9.0"
 implementation "com.squareup.retrofit2:converter-gson:2.9.0"
 implementation "com.squareup.okhttp3:logging-interceptor:4.9.0"

 // Coroutines
 implementation "org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.1"
 implementation "org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.1"

 // Lifecycle components (ViewModel, LiveData)
 implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.1"
}

```

```

implementation "androidx.lifecycle:lifecycle-livedata-ktx:2.6.1"
implementation "androidx.lifecycle:lifecycle-runtime-ktx:2.6.1"

// RecyclerView dan CardView
implementation "androidx.recyclerview:recyclerview:1.3.1"
implementation "androidx.cardview:cardview:1.0.0"

// Glide (untuk memuat gambar dari URL)
implementation 'com.github.bumptech.glide:glide:4.15.1'
annotationProcessor 'com.github.bumptech.glide:compiler:4.15.1'
}

```

Sinkronkan proyek Gradle.

5. **Buat Model Data** ( `Movie.kt` , `MovieResponse.kt` ) di package `data.model` seperti yang dijelaskan di bagian 6.2.
6. **Buat Antarmuka Service API** ( `MovieApiService.kt` ) di package `network` seperti yang dijelaskan di bagian 6.2.
7. **Buat** `RetrofitClient.kt` di package `network` seperti yang dijelaskan di bagian 6.2.
8. **Buat** `Repository` ( `MovieRepository.kt` ) di package `data.repository` seperti yang dijelaskan di bagian 6.2.
9. **Buat** `ViewModel` ( `MovieViewModel.kt` , `MovieViewModelFactory.kt` ) di package `ui.movie` seperti yang dijelaskan di bagian 6.2.
10. **Desain** `activity_main.xml` :  
 Tambahkan `RecyclerView` dan `TextView` untuk pesan error.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
 xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 tools:context=".MainActivity">

 <androidx.recyclerview.widget.RecyclerView
 android:id="@+id/recyclerViewMovies"

```

```

 android:layout_width="0dp"
 android:layout_height="0dp"
 app:layout_constraintBottom_toBottomOf="parent"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toTopOf="parent"
 tools:listitem="@layout/item_movie" />

 <TextView
 android:id="@+id/errorMessageTextView"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:textColor="@android:color/holo_red_dark"
 android:textSize="16sp"
 android:visibility="gone"
 app:layout_constraintBottom_toBottomOf="parent"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toTopOf="parent"
 tools:text="Error: Gagal mengambil data" />

 </androidx.constraintlayout.widget.ConstraintLayout>

```

11. **Buat file layout untuk setiap item** `RecyclerView` ( `item_movie.xml` ) seperti yang dijelaskan di bagian 6.3.
12. **Buat** `Adapter` **untuk** `RecyclerView` ( `MovieAdapter.kt` ) di package `ui.movie` seperti yang dijelaskan di bagian 6.3.
13. **Modifikasi** `MainActivity.kt` :

```

package com.example.myapp

import android.os.Bundle
import android.view.View
import android.widget.TextView
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.myapp.network.RetrofitClient

```

```
import com.example.myapp.data.repository.MovieRepository
import com.example.myapp.ui.movie.MovieAdapter
import com.example.myapp.ui.movie.MovieViewModel
import com.example.myapp.ui.movie.MovieViewModelFactory

class MainActivity : AppCompatActivity() {

 private val movieViewModel: MovieViewModel by viewModels {
 MovieViewModelFactory(MovieRepository(RetrofitClient.instance))
 }

 private lateinit var movieAdapter: MovieAdapter
 private lateinit var recyclerView: RecyclerView
 private lateinit var errorMessageTextView: TextView

 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)

 recyclerView = findViewById(R.id.recyclerViewMovies)
 errorMessageTextView = findViewById(R.id.errorMessageTextView)

 movieAdapter = MovieAdapter()
 recyclerView.layoutManager = LinearLayoutManager(this)
 recyclerView.adapter = movieAdapter

 movieViewModel.movies.observe(this) { movies ->
 movies?.let { movieAdapter.submitList(it) }
 }

 movieViewModel.errorMessage.observe(this) { message ->
 errorMessageTextView.text = message
 errorMessageTextView.visibility = if (message.isNullOrEmpty())
 View.GONE else View.VISIBLE
 }

 // Panggil API untuk mengambil film
 // Ganti "API_KEY_ANDA_DI_SINI" dengan kunci API TMDB Anda yang
 // sebenarnya
 movieViewModel.fetchPopularMovies("API_KEY_ANDA_DI_SINI")
 }
}
```

14. Jalankan aplikasi. Pastikan Anda memiliki koneksi internet. Anda akan melihat daftar film populer ditampilkan di `RecyclerView`. Coba matikan koneksi internet Anda dan jalankan kembali aplikasi untuk melihat penanganan error.

Bagian ini telah membahas secara mendalam tentang bagaimana mengintegrasikan aplikasi Android Anda dengan RESTful API menggunakan Retrofit, mengurai data JSON, dan menampilkannya di UI. Kemampuan untuk berinteraksi dengan layanan backend adalah keterampilan yang sangat penting dalam pengembangan aplikasi modern.

## Bagian 7: Kolaborasi dan Distribusi

### Tujuan Pembelajaran

Setelah menyelesaikan bagian ini, Anda diharapkan mampu:

- Memahami pentingnya sistem kontrol versi (VCS) seperti Git dalam pengembangan perangkat lunak.
- Menggunakan Git dan GitHub untuk mengelola kode sumber dan berkolaborasi dalam tim.
- Melakukan operasi Git dasar seperti `clone`, `add`, `commit`, `push`, `pull`, `branch`, dan `merge`.
- Memahami proses build aplikasi Android, termasuk perbedaan antara build debug dan release.
- Menandatangani (sign) file APK untuk distribusi.
- Mempersiapkan aplikasi untuk rilis di Google Play Store.

### 7.1 Git dan GitHub untuk Kolaborasi Tim

Dalam pengembangan aplikasi modern, terutama dalam tim, mengelola perubahan kode sumber adalah hal yang krusial. **Git** adalah sistem kontrol versi terdistribusi yang paling populer, memungkinkan Anda melacak perubahan dalam kode Anda, kembali ke versi sebelumnya, dan berkolaborasi dengan pengembang lain secara efisien. **GitHub** adalah platform berbasis web yang menyediakan hosting untuk repositori Git, serta fitur-fitur kolaborasi seperti `pull requests` dan `issue tracking`.

#### 7.1.1 Konsep Dasar Git

- **Repository (Repo):** Direktori yang berisi semua file proyek Anda, bersama dengan riwayat revisi setiap file.
- **Commit:** Snapshot dari proyek Anda pada waktu tertentu. Setiap commit memiliki pesan yang menjelaskan perubahan yang dilakukan.
- **Branch:** Salinan independen dari repositori Anda. Digunakan untuk mengembangkan fitur baru atau memperbaiki bug tanpa memengaruhi kode utama.
- **Merge:** Menggabungkan perubahan dari satu branch ke branch lain.
- **Clone:** Membuat salinan lokal dari repositori jarak jauh (remote repository).
- **Pull:** Mengambil perubahan terbaru dari repositori jarak jauh dan menggabungkannya ke repositori lokal Anda.

- **Push:** Mengirim perubahan dari repositori lokal Anda ke repositori jarak jauh.

### 7.1.2 Alur Kerja Git Dasar

#### 1. Inisialisasi Repositori (Lokal):

Jika Anda memulai proyek baru, navigasikan ke direktori proyek Anda di terminal dan inisialisasi repositori Git:

```
git init
```

#### 2. Menambahkan File ke Staging Area:

Sebelum melakukan commit, Anda perlu menambahkan file yang berubah ke staging area. Ini memberitahu Git file mana yang ingin Anda sertakan dalam commit berikutnya.

```
git add nama_file.kt
git add . # Menambahkan semua perubahan di direktori saat ini
```

#### 3. Melakukan Commit:

Setelah file berada di staging area, Anda dapat melakukan commit. Berikan pesan commit yang deskriptif.

```
git commit -m "pesan commit yang deskriptif"
```

#### 4. Menghubungkan ke Repositori Jarak Jauh (GitHub):

Jika Anda memiliki repositori di GitHub, Anda perlu menghubungkan repositori lokal Anda dengannya.

```
git remote add origin <URL_repositori_GitHub_And>
```

#### 5. Mendorong Perubahan ke GitHub:

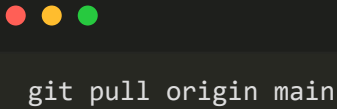
Setelah melakukan commit, Anda dapat mendorong perubahan Anda ke repositori jarak jauh (biasanya `origin` dan branch `main` atau `master`).



```
git pull origin main
```

#### 6. Mengambil Perubahan dari GitHub:

Sebelum memulai pekerjaan atau secara berkala, ambil perubahan terbaru dari repositori jarak jauh.

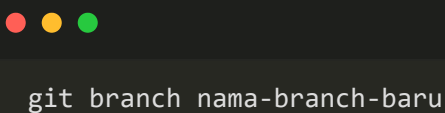


```
git pull origin main
```

### 7.1.3 Branching dan Merging

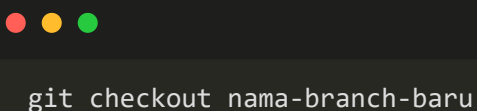
Branching adalah fitur kuat Git yang memungkinkan pengembangan paralel. Setiap fitur baru atau perbaikan bug harus dikembangkan di branch terpisah.

#### 1. Membuat Branch Baru:



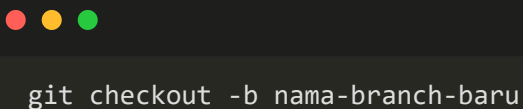
```
git branch nama-branch-baru
```

#### 2. Beralih ke Branch:



```
git checkout nama-branch-baru
```

Atau, untuk membuat dan langsung beralih:



```
git checkout -b nama-branch-baru
```

#### 3. Menggabungkan Branch:

Setelah selesai mengembangkan fitur di branch Anda, beralihlah kembali ke branch utama (misalnya, `main`) dan gabungkan branch fitur Anda.

```
git checkout main
git merge nama-branch-baru
```

#### 4. Menghapus Branch:

Setelah digabungkan, Anda dapat menghapus branch fitur.

```
git branch -d nama-branch-baru
```

### 7.1.4 Pull Requests (GitHub)

**Pull Request** (PR) adalah cara untuk memberitahu orang lain tentang perubahan yang telah Anda dorong ke branch di repositori. Ini memungkinkan pengembang lain untuk meninjau perubahan Anda, memberikan komentar, dan menyarankan modifikasi sebelum perubahan tersebut digabungkan ke branch utama.

#### Alur Kerja Pull Request:

1. Buat branch baru untuk fitur/bugfix Anda.
2. Lakukan perubahan dan commit secara berkala di branch tersebut.
3. Dorong branch Anda ke GitHub ( `git push origin nama-branch-baru` ).
4. Buka **Pull Request** di GitHub dari branch Anda ke branch utama (misalnya, `main` ).
5. Diskusikan perubahan, lakukan revisi jika diperlukan.
6. Setelah disetujui, gabungkan **Pull Request** ke branch utama.

### 7.2 Proses Build Aplikasi Android dan Signing APK

Setelah mengembangkan aplikasi, langkah selanjutnya adalah membangunnya menjadi file APK (Android Package Kit) yang dapat diinstal di perangkat Android. Ada dua jenis build utama:

#### 7.2.1 Build Debug vs. Release

- **Debug Build:** Digunakan selama pengembangan. APK debug ditandatangani dengan kunci debug yang dihasilkan secara otomatis oleh Android Studio. Ini memungkinkan Anda untuk menginstal dan men-debug aplikasi di perangkat atau emulator tanpa perlu konfigurasi tambahan. APK debug tidak dapat didistribusikan ke Google Play Store.
- **Release Build:** Digunakan untuk distribusi ke pengguna. APK release harus ditandatangani dengan kunci rilis (release key) Anda sendiri. Kunci ini adalah identitas unik aplikasi Anda dan harus dijaga kerahasiaannya. Jika Anda kehilangan kunci rilis, Anda tidak akan dapat memperbarui aplikasi Anda di Google Play Store.

### 7.2.2 Menandatangani (Signing) APK

Menandatangani aplikasi adalah langkah keamanan penting yang memastikan bahwa pembaruan aplikasi berasal dari pengembang yang sama. Proses ini melibatkan penggunaan **keystore** dan **key alias**.

#### Langkah-langkah Menandatangani APK (di Android Studio):

1. Di Android Studio, pergi ke **Build** -> **Generate Signed Bundle / APK...**.
2. Pilih **Android App Bundle** atau **APK** (pilih APK jika Anda ingin file APK langsung).
3. Klik **Next**.
4. **Key store path:**
  - Jika Anda belum memiliki **keystore**, klik **Create new...**.
    - **Key store path:** Pilih lokasi untuk menyimpan file **.jks** Anda.
    - **Password:** Buat kata sandi untuk **keystore** Anda.
    - **Key alias:** Beri nama alias untuk kunci Anda (misalnya, **my\_app\_key**).
    - **Password:** Buat kata sandi untuk kunci Anda (bisa sama dengan **keystore** password).
    - **Validity (years):** Atur validitas kunci (disarankan 25 tahun atau lebih).
    - Isi informasi sertifikat (Nama, Unit Organisasi, Organisasi, Kota, Provinsi, Kode Negara).
    - Klik **OK**.
  - Jika Anda sudah memiliki **keystore**, pilih **Choose existing...** dan navigasikan ke file **.jks** Anda.
5. Masukkan **Key store password** dan **Key alias password**.
6. Klik **Next**.
7. Pilih **build variants** ( **release** ).
8. Pilih **Signature Versions** (V1 dan V2 disarankan untuk kompatibilitas).
9. Klik **Finish**.

Android Studio akan membangun APK yang ditandatangani dan menyimpannya di direktori **app/release/** proyek Anda.

### 7.3 Mempersiapkan Aplikasi untuk Rilis di Google Play Store

Setelah Anda memiliki APK yang ditandatangani, Anda dapat mendistribusikannya ke pengguna. Cara paling umum adalah melalui Google Play Store.

#### Langkah-langkah Umum untuk Rilis di Google Play Store:

1. **Buat Akun Developer Google Play:** Ini memerlukan biaya pendaftaran satu kali.
2. **Siapkan Aset Aplikasi:**
  - **Ikon Aplikasi:** Ikon peluncur resolusi tinggi (adaptif).
  - **Screenshot:** Screenshot aplikasi yang menarik untuk berbagai ukuran perangkat.
  - **Feature Graphic:** Gambar banner untuk halaman listing aplikasi Anda.
  - **Video Promosi (Opsional):** Link YouTube untuk video promosi.
3. **Tulis Deskripsi Aplikasi:**

- **Short Description:** Deskripsi singkat yang menarik (maks 80 karakter).
  - **Full Description:** Deskripsi lengkap fitur dan manfaat aplikasi Anda (maks 4000 karakter).
4. **Konfigurasi Aplikasi di Google Play Console:**
- **Buat Aplikasi Baru:** Di Google Play Console, buat aplikasi baru.
  - **Isi Detail Listing Toko:** Nama aplikasi, deskripsi, kategori, tag, informasi kontak.
  - **Unggah Aset:** Unggah ikon, screenshot, dan grafik fitur.
  - **Unggah APK/App Bundle:** Unggah file APK atau Android App Bundle yang telah ditandatangani.
  - **Konfigurasi Negara/Wilayah:** Pilih negara tempat aplikasi Anda akan tersedia.
  - **Harga & Distribusi:** Tentukan apakah aplikasi gratis atau berbayar.
  - **Content Rating:** Lengkapi kuesioner rating konten.
  - **Privacy Policy:** Berikan URL kebijakan privasi Anda.
5. **Uji Aplikasi:** Lakukan pengujian menyeluruh pada berbagai perangkat dan versi Android.
6. **Luncurkan Aplikasi:** Setelah semua konfigurasi selesai dan pengujian berhasil, Anda dapat meluncurkan aplikasi Anda ke jalur rilis (internal test, closed test, open test, production).

### Android App Bundle (AAB):

Google sangat merekomendasikan penggunaan **Android App Bundle (AAB)** daripada APK untuk distribusi di Google Play Store. AAB adalah format publikasi yang mencakup semua kode dan sumber daya terkompilasi aplikasi Anda, tetapi menyerahkan pembuatan APK dan penandatanganan ke Google Play. Ini memungkinkan Google Play untuk mengoptimalkan pengiriman aplikasi ke perangkat pengguna, menghasilkan ukuran unduhan yang lebih kecil.

## Praktikum 7.1: Mengelola Proyek dengan Git dan GitHub

**Tujuan:** Menginisialisasi repositori Git lokal, menghubungkannya ke GitHub, melakukan commit, dan mendorong perubahan.

### Langkah-langkah:

#### 1. Buat Repositori Baru di GitHub:

- Buka GitHub.com dan login.
- Klik tombol **New** untuk membuat repositori baru.
- Beri nama repositori (misalnya, **MyAndroidAppSemester**).
- Pilih **Public** atau **Private**.
- **Jangan** centang **Add a README file**, **Add .gitignore**, atau **Choose a license** untuk saat ini (kita akan menambahkannya secara manual).
- Klik **Create repository**.
- Setelah repositori dibuat, Anda akan melihat instruksi untuk menghubungkan repositori lokal. Salin URL repositori (misalnya, **https://github.com/username/MyAndroidAppSemester.git**).

#### 2. Inisialisasi Git di Proyek Android Studio Anda:

- Buka proyek Android Studio Anda (gunakan proyek dari praktikum sebelumnya atau buat yang baru).

- Buka terminal di Android Studio ( **View** -> **Tool Windows** -> **Terminal** ).
- Pastikan Anda berada di direktori root proyek Anda.
- Inisialisasi Git:

```
git init
```

### 3. **Buat File** **.gitignore** :

File **.gitignore** memberitahu Git file atau direktori mana yang harus diabaikan (tidak dilacak). Ini penting untuk file yang dihasilkan secara otomatis (seperti **build/** , **.idea/** , **local.properties** ) yang tidak perlu disimpan di kontrol versi.

- Buat file baru bernama **.gitignore** di direktori root proyek Anda.
- Isi dengan konten standar untuk proyek Android (Anda bisa mencari "android .gitignore template" online atau menggunakan yang berikut):

```
.gitignore
/local.properties
/.idea/
.DS_Store
/build
/captures
.externalNativeBuild
.cxx
local.properties
*.iml
.gradle/
build/
captures/
.externalNativeBuild/
.cxx/
*.iml
.idea/
!/.idea/runConfigurations
.idea/libraries
.idea/modules.xml
.idea/workspace.xml
.idea/codeStyles
.idea/gradle.xml
.idea/jarRepositories.xml
```

```
.idea/render.experimental.xml
.idea/vcs.xml
.idea/deploymentTargetDropDown.xml
.idea/compiler.xml
.idea/kotlinc.xml
.idea/assetWizardSettings.xml
.idea/navEditor.xml
.idea/migrations.xml
.idea/resourceManager.xml
.idea/appInsights
.idea/compose-rules.xml
.idea/gms_oss_licenses.xml
.idea/ksp.xml
.idea/material_theme_project_settings.xml
.idea/room_settings.xml
.idea/shelf
.idea/watcherTasks.xml
.idea/caches
.idea/libraries
.idea/misc.xml
.idea/modules.xml
.idea/vcs.xml
.idea/workspace.xml
.idea/gradle.xml
.idea/jarRepositories.xml
.idea/render.experimental.xml
.idea/deploymentTargetDropDown.xml
.idea/compiler.xml
.idea/kotlinc.xml
.idea/assetWizardSettings.xml
.idea/navEditor.xml
.idea/migrations.xml
.idea/resourceManager.xml
.idea/appInsights
.idea/compose-rules.xml
.idea/gms_oss_licenses.xml
.idea/ksp.xml
.idea/material_theme_project_settings.xml
.idea/room_settings.xml
.idea/shelf
.idea/watcherTasks.xml
.idea/caches
*.apk
*.aab
```

```
*.aar
.ap_
*.dex
*.class
*.jar
*.so
*.jnilib
*.dll
*.dylib
*.obj
*.o
*.s
*.log
*.txt
*.tmp
*.bak
*.swp
*.swo
*.orig
*.rej
*.diff
*.patch
*.iml
*.ipr
*.iws
.gradle
build
local.properties
.DS_Store
.idea/workspace.xml
.idea/libraries
.idea/modules.xml
.idea/gradle.xml
.idea/jarRepositories.xml
.idea/render.experimental.xml
.idea/deploymentTargetDropDown.xml
.idea/compiler.xml
.idea/kotlinc.xml
.idea/assetWizardSettings.xml
.idea/navEditor.xml
.idea/migrations.xml
.idea/resourceManager.xml
.idea/appInsights
.idea/compose-rules.xml
```

```
.idea/gms_oss_licenses.xml
.idea/ksp.xml
.idea/material_theme_project_settings.xml
.idea/room_settings.xml
.idea/shelf
.idea/watcherTasks.xml
.idea/caches
```

#### 4. Tambahkan dan Commit Perubahan Awal:

```
git add .
git commit -m "Initial commit: Project setup and .gitignore"
```

#### 5. Hubungkan Repositori Lokal ke GitHub:

Ganti `<URL_repositori_GitHub_Anda>` dengan URL yang Anda salin dari GitHub.

```
git remote add origin <URL_repositori_GitHub_Anda>
```

#### 6. Dorong Perubahan ke GitHub:

```
git push -u origin main
```

Anda mungkin akan diminta untuk memasukkan username dan password GitHub Anda.

#### 7. Verifikasi di GitHub:

Refresh halaman repositori Anda di GitHub. Anda akan melihat file proyek Anda telah diunggah.

### Praktikum 7.2: Membuat dan Menandatangani APK Release

**Tujuan:** Membangun APK release yang ditandatangani yang siap untuk distribusi.

#### Langkah-langkah:

##### 1. Buka Proyek Android Studio Anda.

## 2. Generate Signed Bundle / APK:

- Di menu atas, klik **Build** -> **Generate Signed Bundle / APK...**
- Pilih **APK** dan klik **Next**.

## 3. Buat Keystore Baru (jika belum ada):

- Jika ini pertama kalinya Anda membuat APK yang ditandatangani, klik **Create new...**
- **Key store path:** Klik ikon folder dan pilih lokasi yang aman di komputer Anda untuk menyimpan file **.jks** (misalnya, di luar direktori proyek Anda). Beri nama file (misalnya, **my\_release\_key.jks**).
- **Password:** Masukkan kata sandi yang kuat untuk **keystore** Anda dan konfirmasi.
- **Alias:** Masukkan alias untuk kunci Anda (misalnya, **my\_app\_alias**).
- **Password:** Masukkan kata sandi yang kuat untuk alias kunci Anda dan konfirmasi (bisa sama dengan **keystore** password).
- **Validity (years):** Atur ke 25 tahun atau lebih.
- Isi detail sertifikat (Nama Depan & Belakang, Unit Organisasi, Organisasi, Kota, Provinsi, Kode Negara).
- Klik **OK**.

## 4. Pilih Keystore yang Ada (jika sudah ada):

- Jika Anda sudah memiliki **keystore**, pilih **Choose existing...** dan navigasikan ke file **.jks** Anda.
- Masukkan **Key store password** dan **Key alias password**.

## 5. Konfigurasi Build:

- Klik **Next**.
- Pilih **build variant** sebagai **release**.
- Centang **V1 (Jar Signature)** dan **V2 (Full APK Signature)** untuk kompatibilitas terbaik.
- Klik **Finish**.

## 6. Temukan APK yang Ditandatangani:

- Setelah proses build selesai, Android Studio akan menampilkan notifikasi dengan link **locate**. Klik link tersebut.
- Ini akan membuka folder di mana APK release Anda (**app-release.apk**) disimpan (biasanya di **app/release/**).

Anda sekarang memiliki file APK yang ditandatangani dan siap untuk didistribusikan secara manual atau diunggah ke Google Play Store.

Bagian ini telah membekali Anda dengan pengetahuan dan keterampilan penting untuk berkolaborasi dalam pengembangan aplikasi menggunakan Git dan GitHub, serta memahami proses build dan distribusi aplikasi

Android. Ini adalah langkah terakhir sebelum aplikasi Anda dapat diakses oleh pengguna di seluruh dunia.

## Kesimpulan

Modul ini telah membimbing Anda melalui perjalanan komprehensif dalam pengembangan aplikasi Android menggunakan Kotlin, mulai dari dasar-dasar UI responsif, logika bisnis yang kuat dengan Kotlin lanjutan dan coroutines, navigasi aplikasi yang efisien, hingga penyimpanan data lokal dengan Room Database dan integrasi dengan RESTful API. Selain itu, Anda juga telah mempelajari praktik terbaik untuk kolaborasi tim menggunakan Git dan GitHub, serta proses penting dalam mempersiapkan aplikasi untuk distribusi.

Dengan pengetahuan dan keterampilan yang Anda peroleh dari modul ini, Anda kini memiliki fondasi yang kuat untuk membangun aplikasi Android yang kompleks, fungsional, dan siap untuk dunia nyata. Teruslah berlatih, eksplorasi fitur-fitur baru, dan jangan ragu untuk berinovasi. Dunia pengembangan Android terus berkembang, dan pembelajaran berkelanjutan adalah kunci kesuksesan.

Selamat berkarya dan semoga sukses dalam perjalanan pengembangan aplikasi Android Anda!

## Referensi

- [Android Developers Documentation](#)
- [Kotlin Official Documentation](#)
- [The Android Lifecycle](#)
- [Understand Android Intents](#)
- [Fragments overview](#)
- [Navigate with the Navigation component](#)
- [Save data using Room](#)
- [Retrofit](#)
- [Glide](#)
- [Git Documentation](#)
- [GitHub Guides](#)
- [Prepare and release your app](#)